

thesis_project

March 5, 2025



1 Predicting Housing Market Trends Using Data Science

Potential Insights: This dataset provides insights into housing market trends across different regions. I chose this dataset because housing market data reflects economic patterns, consumer behavior, and financial stability. By analyzing price fluctuations, sales trends, and inventory changes, I aim to identify key indicators that predict market shifts. The structured approach from Kaggle competitions will be applied to extract meaningful patterns, focusing on how market dynamics evolve over time.

Additionally, the dataset's inclusion of multiple region types allows for comparisons between national trends and city-specific behaviors, providing an opportunity to explore how different real estate markets respond to economic factors. By leveraging machine learning techniques learned from the competitions, I will analyze potential leading indicators of price changes and market cycles.

1.0.1 Project Scope

Objectives: Identify key trends and patterns in the housing market, predict market fluctuations, and provide actionable insights into real estate trend

Deliverables: Charts, graphs, statistical reports, predictive models, and a final research paper detailing insights and methodologies.

Milestones:

- Data collection and cleaning (Week 1)
- Exploratory Data Analysis (Weeks 2-3)
- Model selection and training (Weeks 4-6)
- Model validation and optimization (Weeks 7-8)
- Final report and presentation (Weeks 9-10)

Tasks:

- Data acquisition and preprocessing
- Exploratory data analysis (EDA)
- Feature engineering and selection
- Model training and validation
- Performance evaluation and optimization
- Presentation of findings

Resources: Python, Pandas, NumPy, Scikit-Learn, Matplotlib, Seaborn, Jupyter Notebook

1.0.2 Research Plan

Techniques and Methods: Time-series analysis, regression models, machine learning techniques, and cross-validation methods. Application: Apply time-series forecasting models to predict housing price trends, use feature engineering techniques to identify key drivers of price fluctuations, and validate results using robust evaluation metrics.

Hypothesis - Hypothesis Statement: Housing market trends can be predicted by analyzing historical price fluctuations, inventory changes, and sales patterns. By applying machine learning techniques, we can identify early indicators of market shifts, allowing for better-informed real estate investment decisions.

2 Imports

```
[1]: # my imports
import wrangle as w
import explore as e
import modeling as m

# ignore warnings
import warnings
warnings.filterwarnings("ignore")
```

imports loaded successfully, awaiting commands...

2.1 # Wrangle

- Raw Data acquired from Redfin
- 1099 rows x 20 columns before cleaning
- Data acquired from Google BigQuery after cleaning
- 1099 rows x 20 columns after cleaning

2.2 ## Prepare

- Clean the data
 - Rename columns
 - Remove nulls
 - Remove extra characters
 - Remove extra spaces

– Fixed data type

```
[2]: redfin_df = w.check_file_exists_gbq("data.csv", "service-account-key.json",  
                                         "iu-thesis-project."  
                                         ↪Redfin_Monthly_Housing_Market_Data.Redfin")
```

CSV file 'data.csv' found locally. Checking data quality...
Detected encoding: ascii
Data appears to be clean.
Data is already clean. No need to clean or upload.

2.3 ### A Brief Look At The Data

```
[3]: redfin_df.head()
```

```
[3]:      region      month_of_period_end  median_sale_price  \  
0  National 2022-07-01 00:00:00+00:00      413000  
1  National 2022-08-01 00:00:00+00:00      407000  
2  National 2022-06-01 00:00:00+00:00      429000  
3  National 2021-08-01 00:00:00+00:00      381000  
4  National 2021-09-01 00:00:00+00:00      377000  
  
      median_sale_price_mom  median_sale_price_yoy  homes_sold  homes_sold_mom  \  
0              -3.6              7.4      530476      -14.4  
1              -1.5              6.9      559150       5.4  
2              -0.7             10.8      619857       4.1  
3              -1.1             16.2      672612      -1.0  
4              -1.0             13.9      638337      -5.1  
  
      homes_sold_yoy  new_listings  new_listings_mom  new_listings_yoy  \  
0          -21.9      705215      -14.3      -11.9  
1          -16.9      622537      -11.7      -13.7  
2          -15.0      822855       6.4       1.0  
3           0.1      721768      -9.8       0.0  
4          -4.7      675008      -6.5      -4.5  
  
      inventory  inventory_mom  inventory_yoy  days_on_market  \  
0      1240506      10.2      27.1      21  
1      1225537      -1.2      26.7      27  
2      1125321      20.7      28.1      18  
3       967307      -0.9     -18.2      17  
4       969546       0.2     -16.1      19  
  
      days_on_market_mom  days_on_market_yoy  average_sale_to_list  \  
0              3              6      101.0  
1              5             10      99.9  
2              1              3     102.2  
3              2             -15     101.6
```

4	2	-11	101.0
---	---	-----	-------

	average_sale_to_list_mom	average_sale_to_list_yoy
0	-1.2	-1.2
1	-1.1	-1.7
2	-0.8	-0.3
3	-0.6	2.4
4	-0.6	1.7

2.4 A Summary Of The Data

```
[4]: e.data_summary(redfin_df)
```

```
data shape: (1099, 20)
```

```
[4]:
```

	data type	#missing	%missing	#unique	\
region	object	0	0.0	7	
month_of_period_end	datetime64[ns, UTC]	0	0.0	157	
median_sale_price	int64	0	0.0	416	
median_sale_price_mom	float64	0	0.0	186	
median_sale_price_yoy	float64	0	0.0	251	
homes_sold	int64	0	0.0	1036	
homes_sold_mom	float64	0	0.0	556	
homes_sold_yoy	float64	0	0.0	490	
new_listings	int64	0	0.0	1062	
new_listings_mom	float64	0	0.0	634	
new_listings_yoy	float64	0	0.0	462	
inventory	int64	0	0.0	1084	
inventory_mom	float64	0	0.0	367	
inventory_yoy	float64	0	0.0	558	
days_on_market	int64	0	0.0	100	
days_on_market_mom	int64	0	0.0	45	
days_on_market_yoy	int64	0	0.0	86	
average_sale_to_list	float64	0	0.0	130	
average_sale_to_list_mom	float64	0	0.0	43	
average_sale_to_list_yoy	float64	0	0.0	99	

	count	mean	\
region	1099	NaN	
month_of_period_end	1099	2018-07-01 19:52:21.401273856+00:00	
median_sale_price	1099.0	407515.013649	
median_sale_price_mom	1099.0	0.610464	
median_sale_price_yoy	1099.0	6.849045	
homes_sold	1099.0	73960.789809	
homes_sold_mom	1099.0	1.37152	
homes_sold_yoy	1099.0	1.94222	
new_listings	1099.0	89187.496815	

new_listings_mom	1099.0	3.776797
new_listings_yoy	1099.0	0.10919
inventory	1099.0	224060.347589
inventory_mom	1099.0	-0.101001
inventory_yoy	1099.0	-4.468062
days_on_market	1099.0	38.658781
days_on_market_mom	1099.0	-0.213831
days_on_market_yoy	1099.0	-3.524113
average_sale_to_list	1099.0	99.071338
average_sale_to_list_mom	1099.0	0.020291
average_sale_to_list_yoy	1099.0	0.301911

	std	min \
region	NaN	NaN
month_of_period_end	NaN	2012-01-01 00:00:00+00:00
median_sale_price	189404.64317	134000.0
median_sale_price_mom	3.657105	-12.8
median_sale_price_yoy	5.72749	-10.7
homes_sold	174049.251215	974.0
homes_sold_mom	18.047561	-40.6
homes_sold_yoy	17.349035	-56.9
new_listings	210707.155307	1010.0
new_listings_mom	29.206833	-59.5
new_listings_yoy	17.261469	-70.1
inventory	541161.876914	1057.0
inventory_mom	9.783868	-39.3
inventory_yoy	23.895725	-60.5
days_on_market	21.157081	5.0
days_on_market_mom	6.726423	-49.0
days_on_market_yoy	12.70981	-69.0
average_sale_to_list	2.213431	92.0
average_sale_to_list_mom	0.510114	-4.1
average_sale_to_list_yoy	1.418228	-10.6

	25% \
region	NaN
month_of_period_end	2015-04-01 00:00:00+00:00
median_sale_price	256500.0
median_sale_price_mom	-1.8
median_sale_price_yoy	3.35
homes_sold	3283.0
homes_sold_mom	-10.7
homes_sold_yoy	-6.5
new_listings	3613.0
new_listings_mom	-13.0
new_listings_yoy	-6.8
inventory	7138.0

inventory_mom	-4.45
inventory_yoy	-17.4
days_on_market	23.0
days_on_market_mom	-2.0
days_on_market_yoy	-8.0
average_sale_to_list	97.7
average_sale_to_list_mom	-0.2
average_sale_to_list_yoy	-0.1

	50%	\
region	NaN	
month_of_period_end	2018-07-01 00:00:00+00:00	
median_sale_price	371000.0	
median_sale_price_mom	0.2	
median_sale_price_yoy	6.5	
homes_sold	5108.0	
homes_sold_mom	0.2	
homes_sold_yoy	1.8	
new_listings	6070.0	
new_listings_mom	-2.5	
new_listings_yoy	0.2	
inventory	12397.0	
inventory_mom	0.4	
inventory_yoy	-7.3	
days_on_market	36.0	
days_on_market_mom	1.0	
days_on_market_yoy	-2.0	
average_sale_to_list	99.1	
average_sale_to_list_mom	-0.1	
average_sale_to_list_yoy	0.3	

	75%		max
region	NaN		NaN
month_of_period_end	2021-10-01 00:00:00+00:00	2025-01-01 00:00:00+00:00	
median_sale_price	525000.0	925000.0	
median_sale_price_mom	2.8	16.4	
median_sale_price_yoy	9.7	31.8	
homes_sold	7373.0	729414.0	
homes_sold_mom	11.55	63.4	
homes_sold_yoy	10.7	127.6	
new_listings	9152.0	844234.0	
new_listings_mom	14.05	150.3	
new_listings_yoy	6.75	257.9	
inventory	22078.0	2181834.0	
inventory_mom	4.55	49.4	
inventory_yoy	4.9	230.0	
days_on_market	49.0	145.0	

days_on_market_mom	4.0	19.0
days_on_market_yoy	3.0	43.0
average_sale_to_list	100.2	110.7
average_sale_to_list_mom	0.3	3.6
average_sale_to_list_yoy	0.9	7.2

```
[5]: redfin_df.info()
```

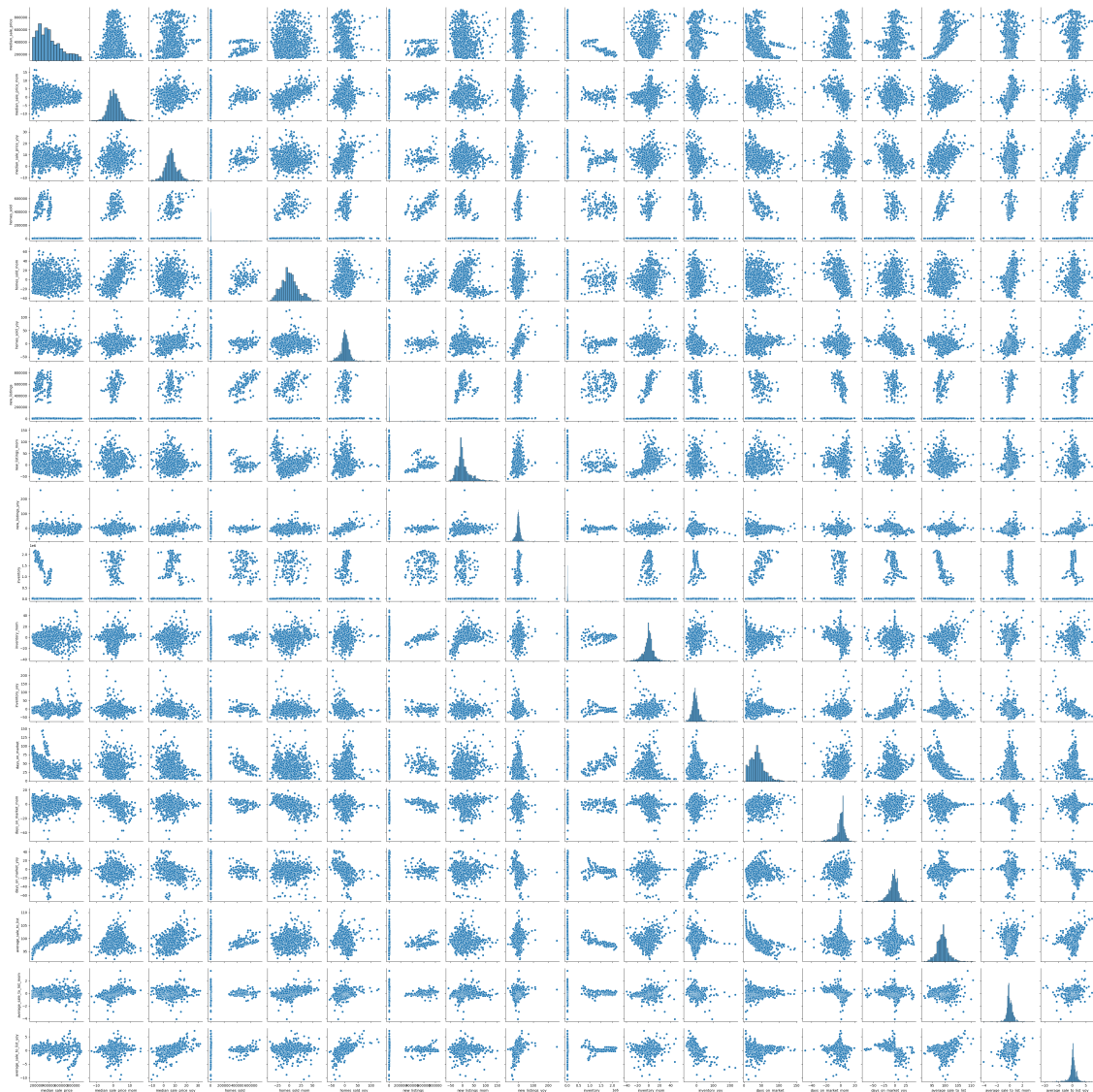
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1099 entries, 0 to 1098
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   region                                1099 non-null   object
1   month_of_period_end                  1099 non-null   datetime64[ns, UTC]
2   median_sale_price                    1099 non-null   int64
3   median_sale_price_mom                1099 non-null   float64
4   median_sale_price_yoy                1099 non-null   float64
5   homes_sold                           1099 non-null   int64
6   homes_sold_mom                       1099 non-null   float64
7   homes_sold_yoy                       1099 non-null   float64
8   new_listings                         1099 non-null   int64
9   new_listings_mom                     1099 non-null   float64
10  new_listings_yoy                     1099 non-null   float64
11  inventory                            1099 non-null   int64
12  inventory_mom                        1099 non-null   float64
13  inventory_yoy                        1099 non-null   float64
14  days_on_market                       1099 non-null   int64
15  days_on_market_mom                   1099 non-null   int64
16  days_on_market_yoy                   1099 non-null   int64
17  average_sale_to_list                 1099 non-null   float64
18  average_sale_to_list_mom             1099 non-null   float64
19  average_sale_to_list_yoy             1099 non-null   float64
dtypes: datetime64[ns, UTC](1), float64(11), int64(7), object(1)
memory usage: 171.8+ KB
```

3 Exploration

```
[6]: # visualized your data
import matplotlib.pyplot as plt
import seaborn as sns

# to view a bit closer the pairplot double click on the plot
sns.pairplot(redfin_df)
```

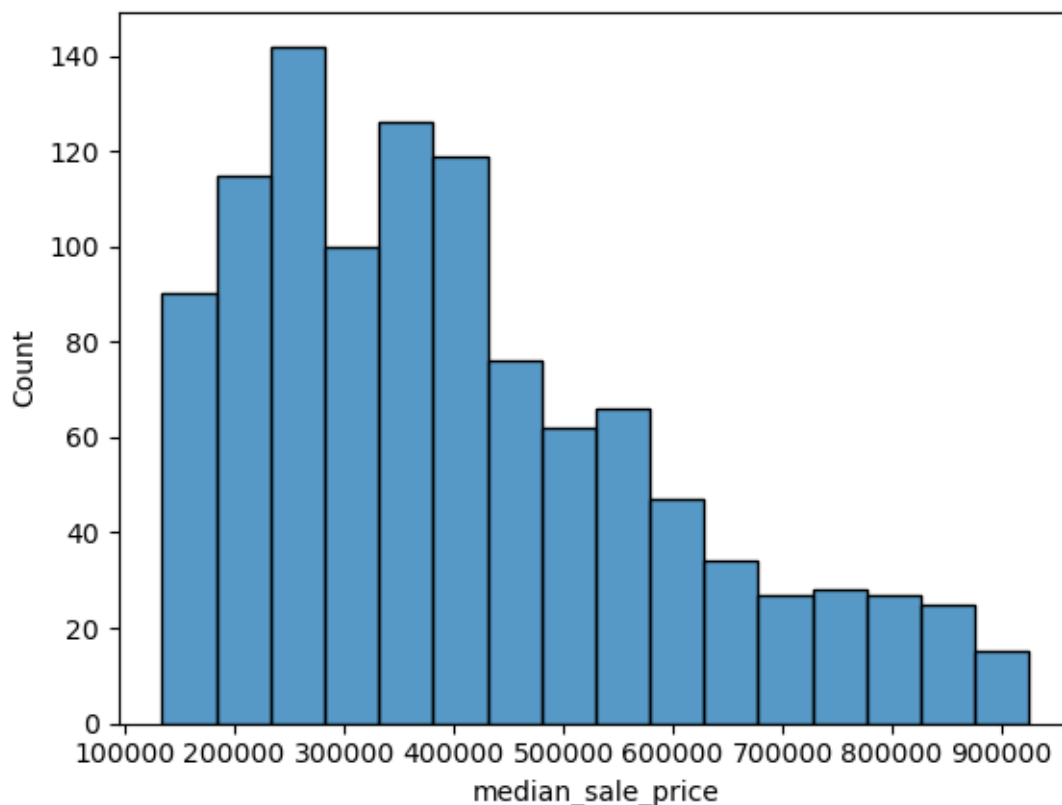
```
[6]: <seaborn.axisgrid.PairGrid at 0x332aa2130>
```



What is the distribution of the Target?

- Note: this question is more of a look at the data, not a stats test question.

```
[7]: # get a histplot of quality
sns.histplot(redfin_df.median_sale_price)
plt.show()
```

Target Justification: median_sale_price

1. Core Predictive Goal
 - Both the Project Proposal (focused on housing market trend prediction) and Literature Review (highlighting house-price forecasting as critical) point to home prices as the principal outcome of interest.
 - As emphasized in Assignment 2, “Accurately predicting real estate house prices is critical for financial institutions, investors, and policymakers.”
2. Data Alignment
 - The Redfin dataset explicitly provides median_sale_price (along with month-over-month and year-over-year variations).
 - In real estate forecasting literature, the median sale price is a standard measure, serving as a robust, central indicator of market conditions.
3. Common Industry Metric
 - Forecasting median sale price is standard practice in both industry and academic settings. It allows analysis of market health and trend over time, region by region.

Histogram Insights

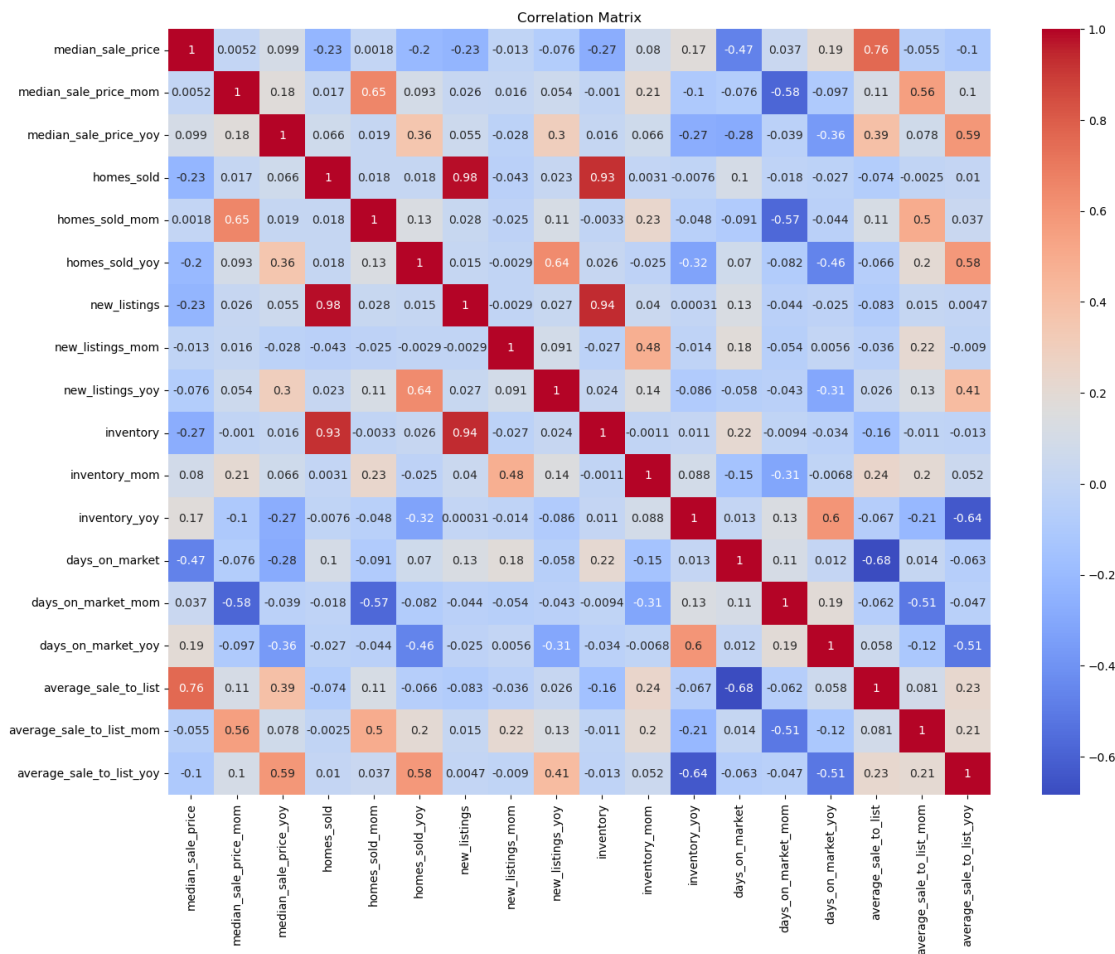
Looking at the histogram of median_sale_price:

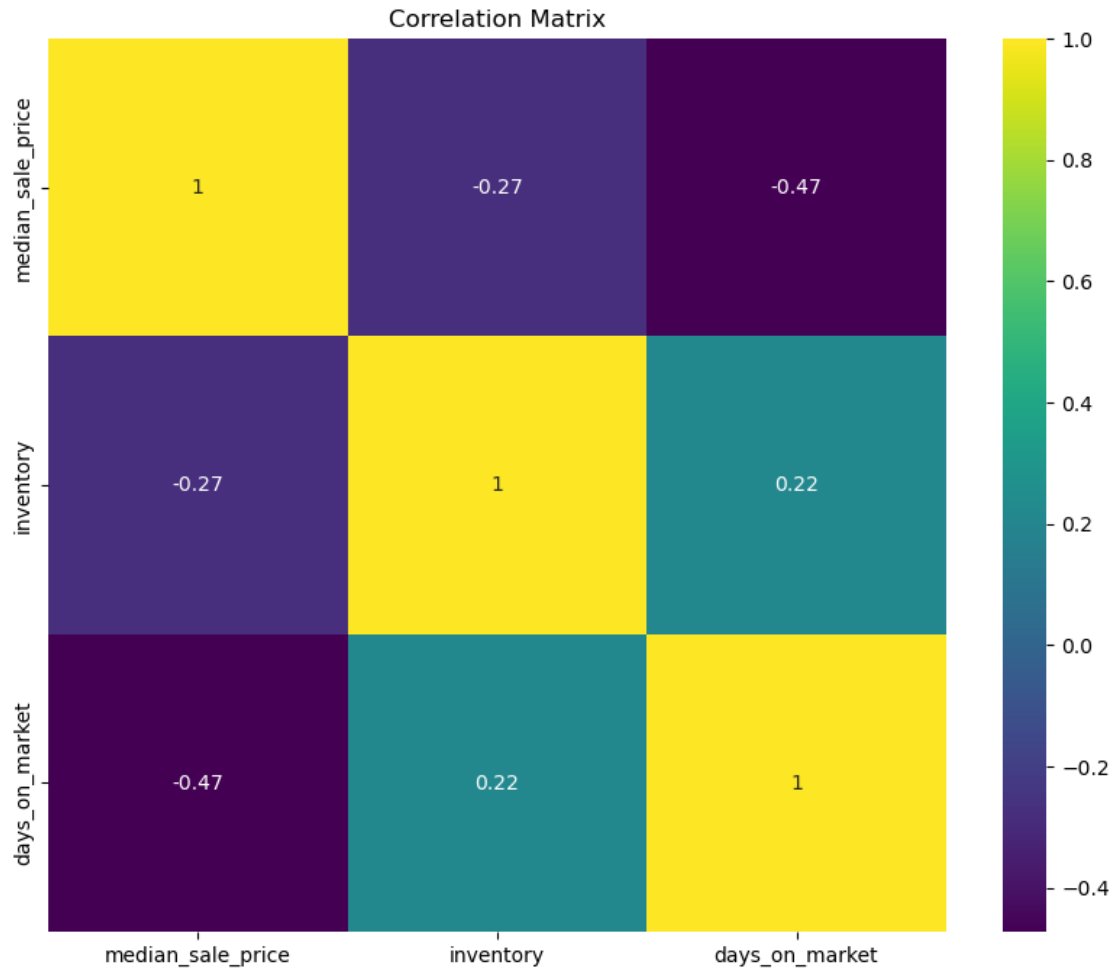
- The distribution peaks roughly in the \$200,000 - \$300,000 range, indicating that many observations cluster around these mid-range house prices.
- The tail extends toward higher price points (up to \$900,000), reflecting that while most houses

fall in the lower-to-mid range, there are notable but fewer properties in higher price brackets. - The shape suggests a right-skew (common in housing data), where a larger number of transactions occur at relatively moderate price levels, and fewer yet higher-priced properties cause the tail to stretch to the right.

```
[8]: # Correlation heatmap for all columns
e.plot_correlation_heatmap(redfin_df)

# Correlation heatmap for a subset of columns
cols_of_interest = ['median_sale_price', 'inventory', 'days_on_market']
e.plot_correlation_heatmap(redfin_df, columns=cols_of_interest, figsize=(10,8),
    cmap='viridis')
```





Correlation Matrix (heatmap) Insights

Notable Correlation Patterns

- Negative Correlation:** median_sale_price vs. days_on_market - Around -0.47 for the current-month relationship - Suggests that when prices are higher, properties tend to sell faster (fewer days on market). Conversely, in slower markets with longer days on market, median prices are typically lower.
- Positive Correlation Between “_yoy” Columns** - You may see strong positive correlations among “year-over-year” metrics—e.g., new_listings_yoy vs. inventory_yoy. - Indicates that increases in new listings over last year often coincide with increases in inventory relative to last year.
- Moderate/Strong Relationship:** median_sale_price and average_sale_to_list - Look for a reddish block in the row/column intersection of median_sale_price and average_sale_to_list (and possibly their _yoy versions). - A higher sale-to-list ratio can coincide with a stronger market, often reflected in higher median prices.
- Month-over-Month (MoM) Correlations** - Some MoM columns (like homes_sold_mom or median_sale_price_mom) may show moderate correlations with each other. - This could imply, for example, that monthly surges in sales might accompany monthly rises in price—or vice versa.
- Days on Market and Homes Sold** - Often, a negative correlation emerges between days_on_market and homes_sold or homes_sold_mom: markets with more successful sales typically see homes spending fewer days on the market.

Interpreting High Correlation Values - Close to +1: Two features move in the same direction nearly every time (e.g., new_listings vs. inventory might be highly correlated if more listings generally lead to more inventory).

- Close to -1: A strong inverse relationship (e.g., higher median_sale_price typically corresponds to fewer days_on_market).

- Close to 0: Little direct linear relationship. There may be other complex or nonlinear relationships, or it may just indicate no meaningful interaction.

Caveats & Next Steps - Correlation Causation: Even if median_sale_price is strongly correlated with inventory, it doesn't confirm a direct cause-effect. External factors like interest rates or broader economic conditions could drive both.

- Outliers & Nonlinearities: Some correlations may appear weaker or stronger if there are a few extreme outliers or nonlinear relationships. Pair plots, scatterplots, or transformations (e.g., log scales) can help clarify.

- Modeling Implications: Strongly correlated variables may lead to multicollinearity in linear models, so we will consider advanced feature selection or regularization (e.g., Ridge/Lasso) if that's an issue.

```
[9]: e.plot_seasonal_decomposition(redfin_df, date_col='month_of_period_end',  
    ↪target_col='median_sale_price', model='multiplicative', period=12)
```



Seasonal Decomposition Insights

1. Observed

- The top series shows a long-term increase in median sale prices from around 2012 to 2024.
- Notice the monthly fluctuations—some months are higher or lower than the smoothed trend, consistent with typical real estate seasonality.

2. Trend

- The middle-top subplot reveals a steady upward slope from roughly \ \$200k to \ \$500k+ over the last decade, reflecting long-term market growth.
- After 2022–2023, the trend appears to plateau or slow, suggesting potential cooling or stabilization in prices.

3. Seasonal

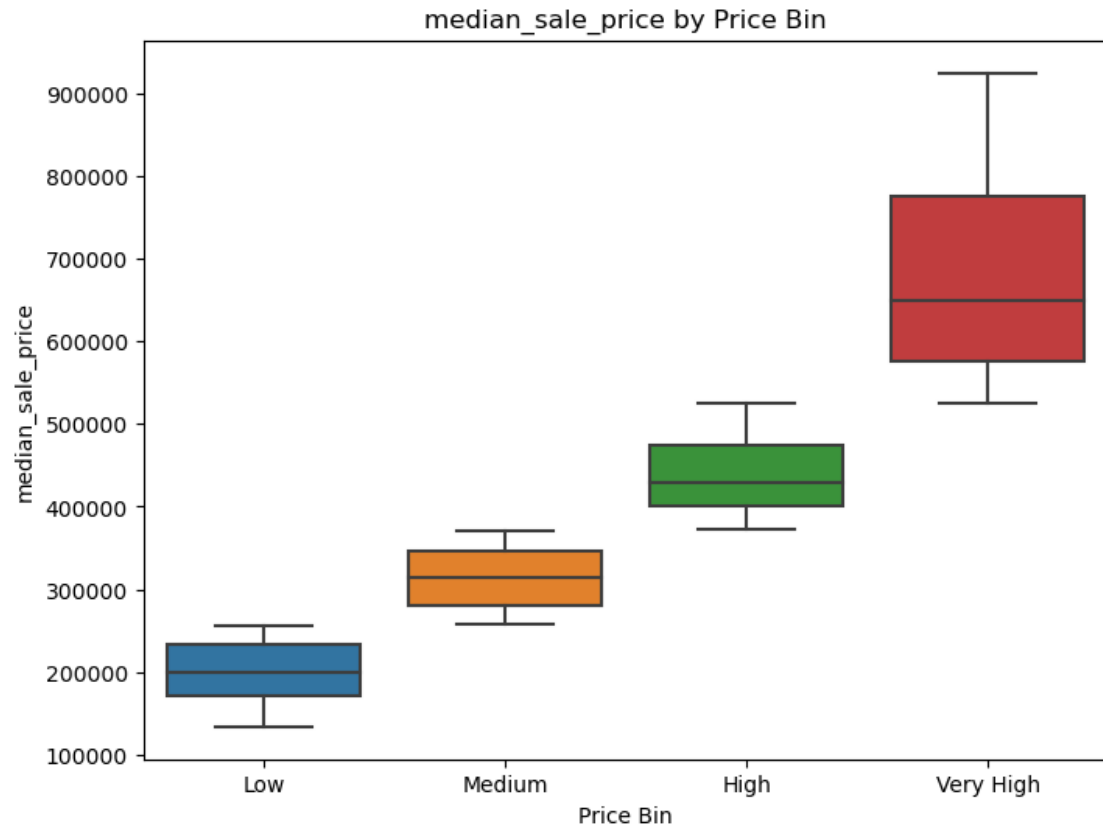
- Because the model is multiplicative, the seasonal factor hovers around 1.0—meaning monthly prices are typically within a few percentage points above or below the overall trend.
- The real estate market often shows seasonal peaks (e.g., spring/summer) and troughs (winter). In a multiplicative model, these bumps are a percentage of the underlying trend, rather than a fixed amount.

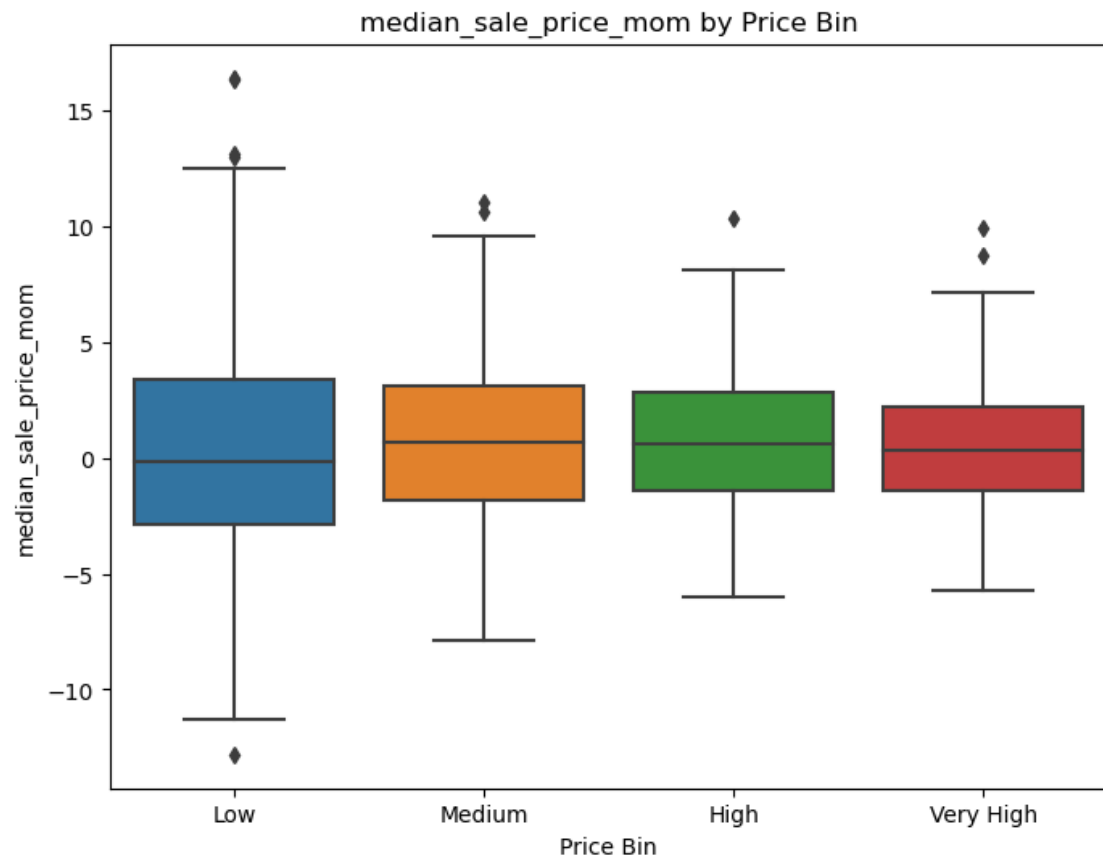
4. Residual

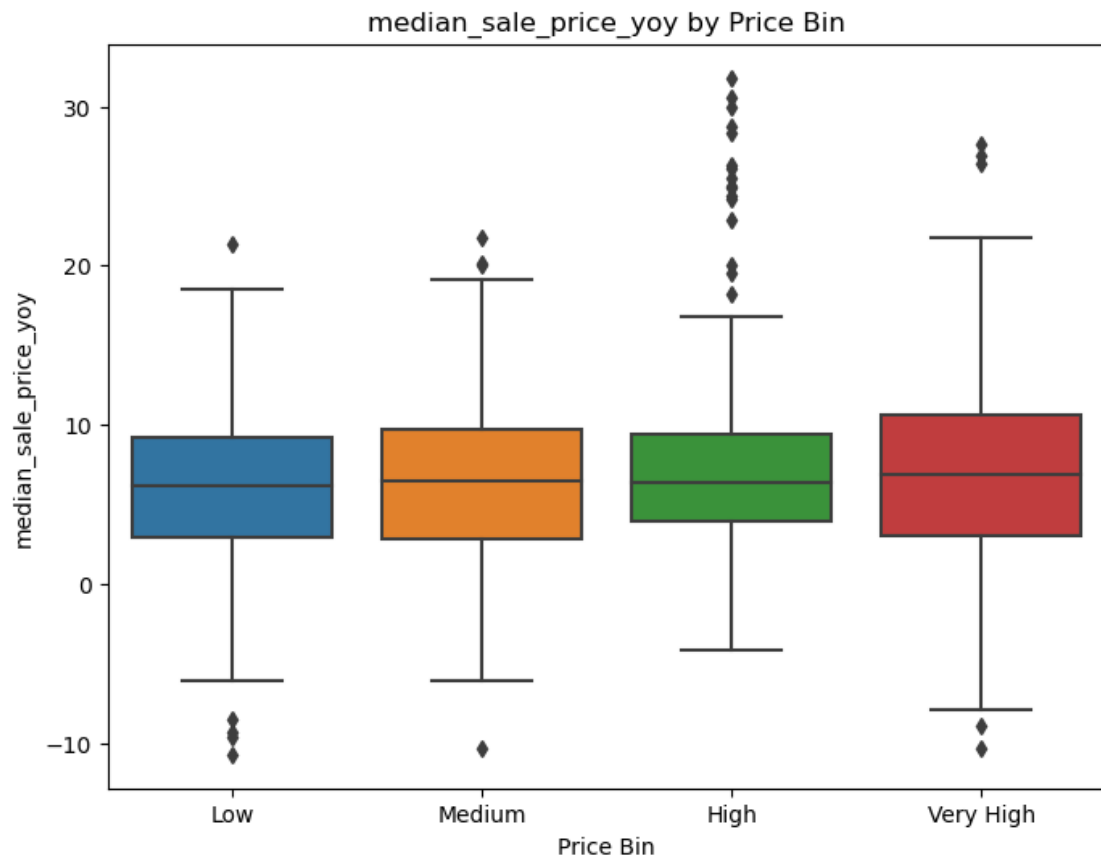
- The bottom subplot shows what’s left after removing the trend and seasonal components.
- If the residual looks mostly random, that indicates the trend+seasonality capture the main structure of the series. Occasional spikes or dips might point to external shocks (e.g., policy changes or unexpected events).

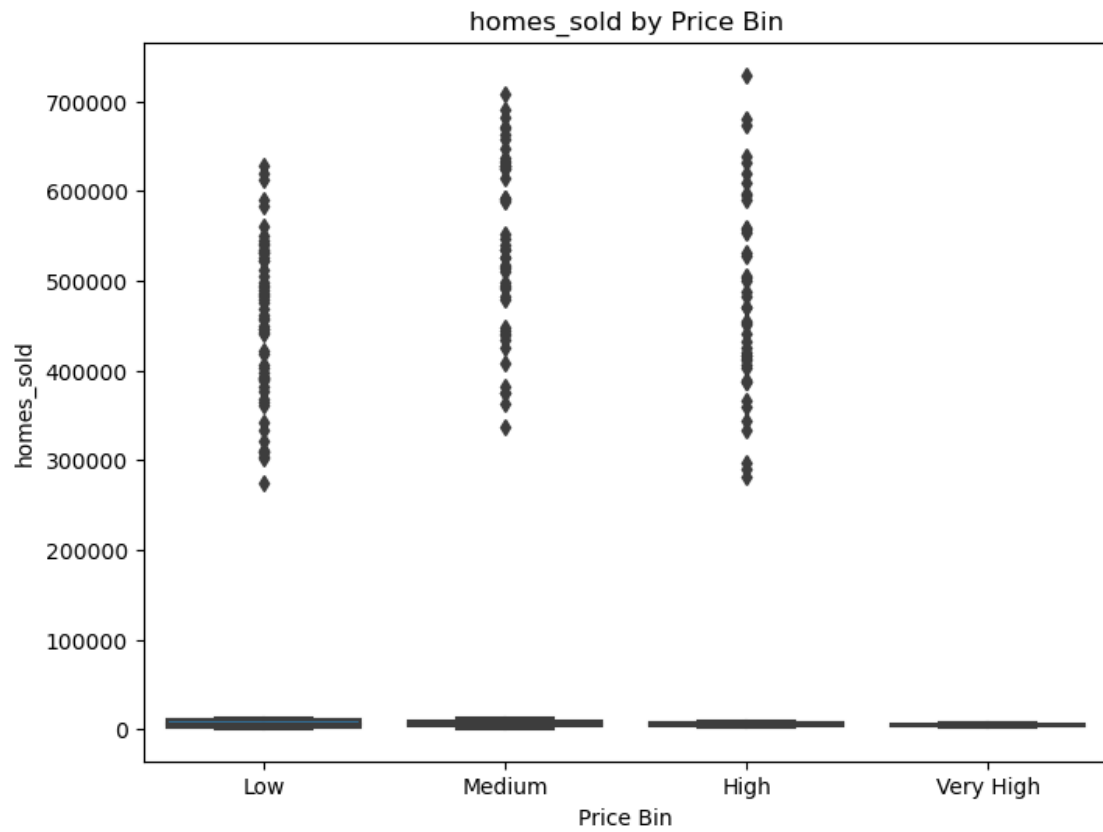
```
[10]: # Identifying numerical and categorical columns
numerical_cols = redfin_df.select_dtypes(include=['int64', 'float64']).columns
categorical_cols = redfin_df.select_dtypes(include=['object', 'bool']).columns
```

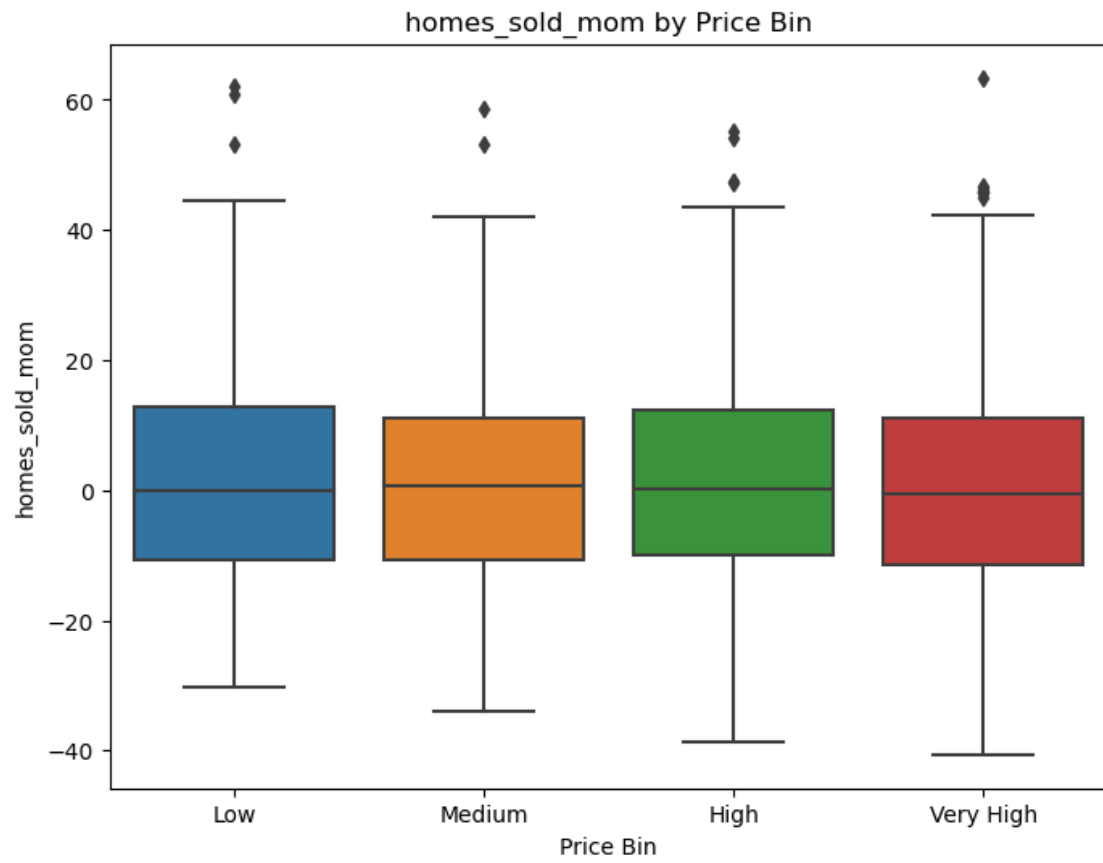
```
[11]: e.bivariate_boxplot(redfin_df, numerical_cols, categorical_cols)
```

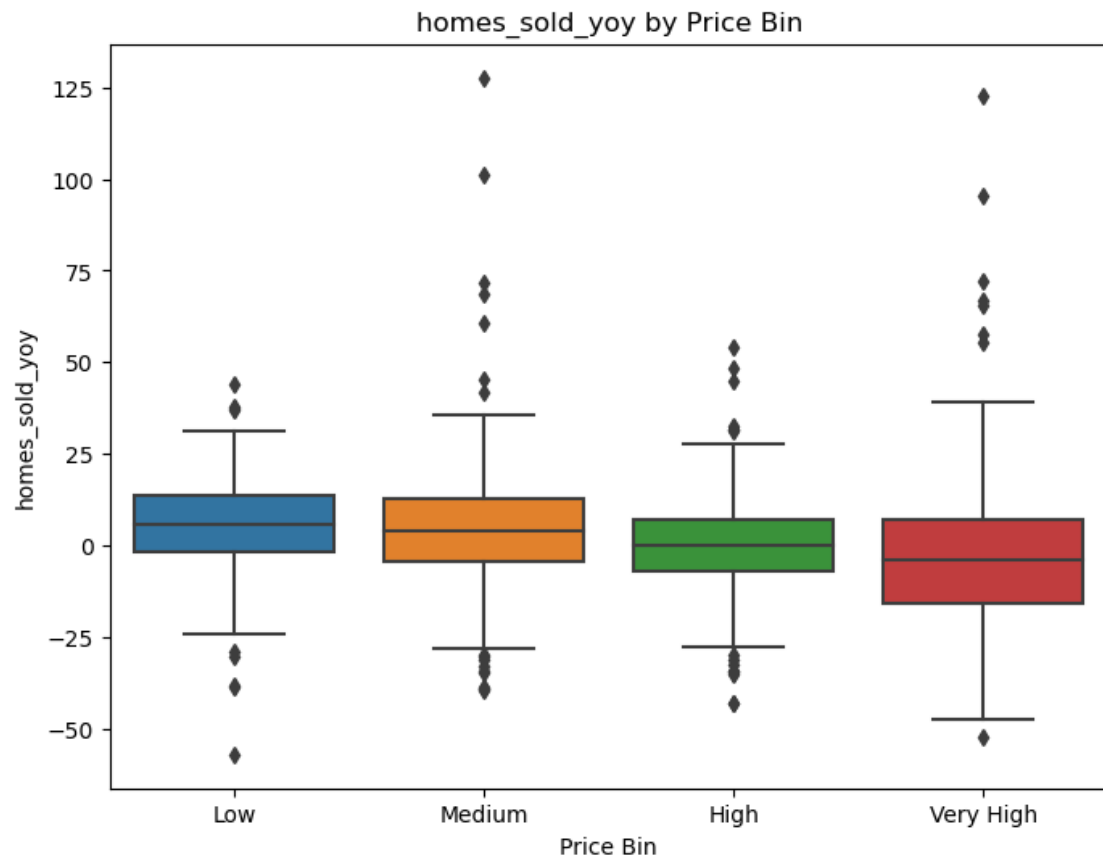


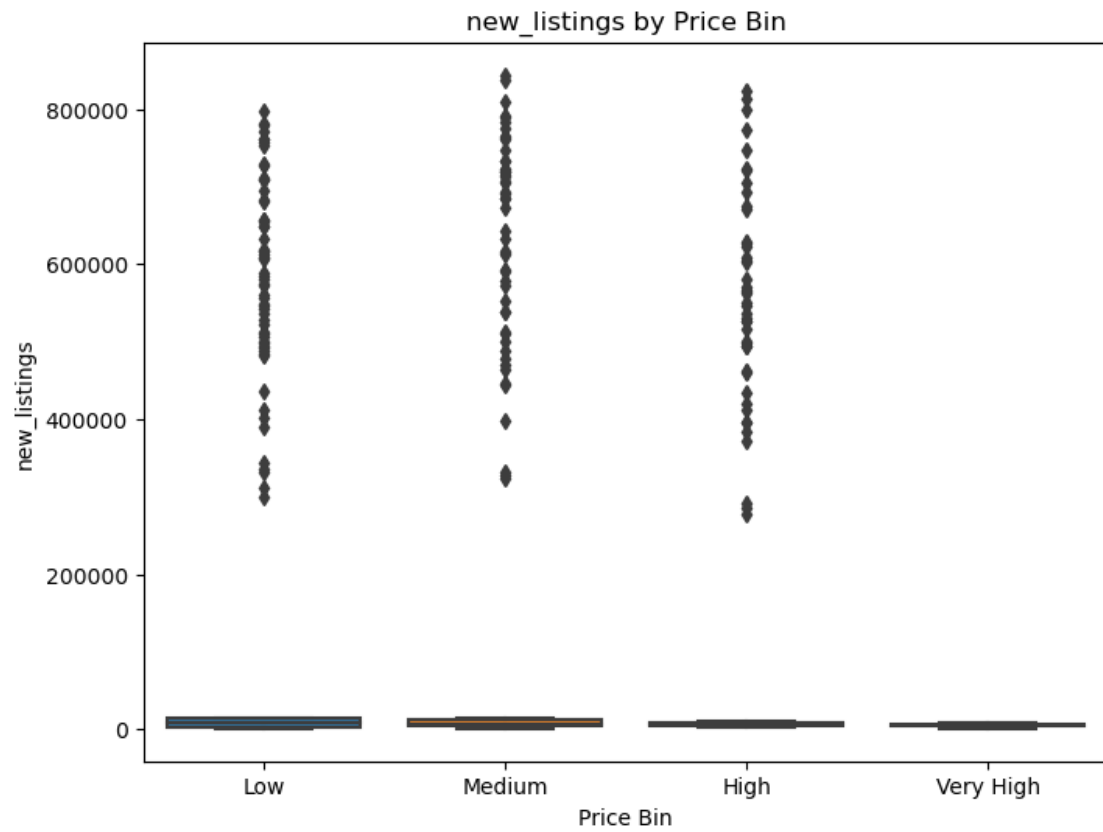


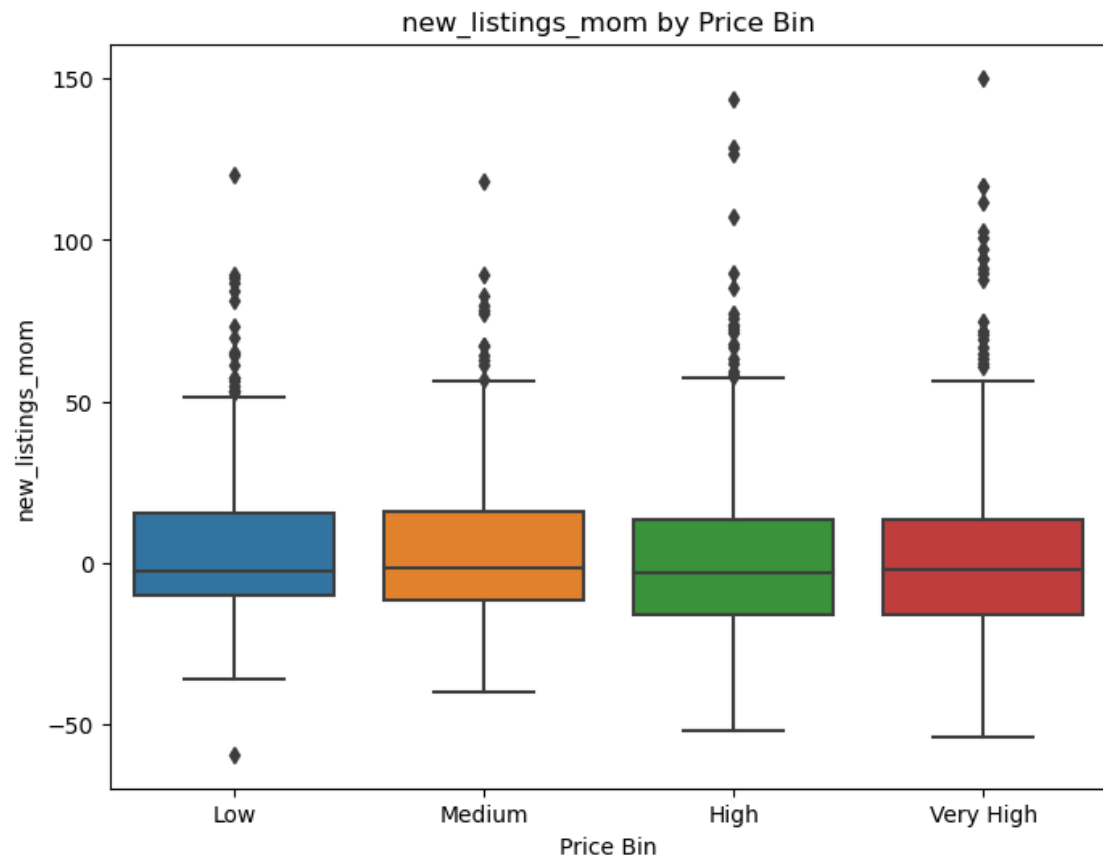


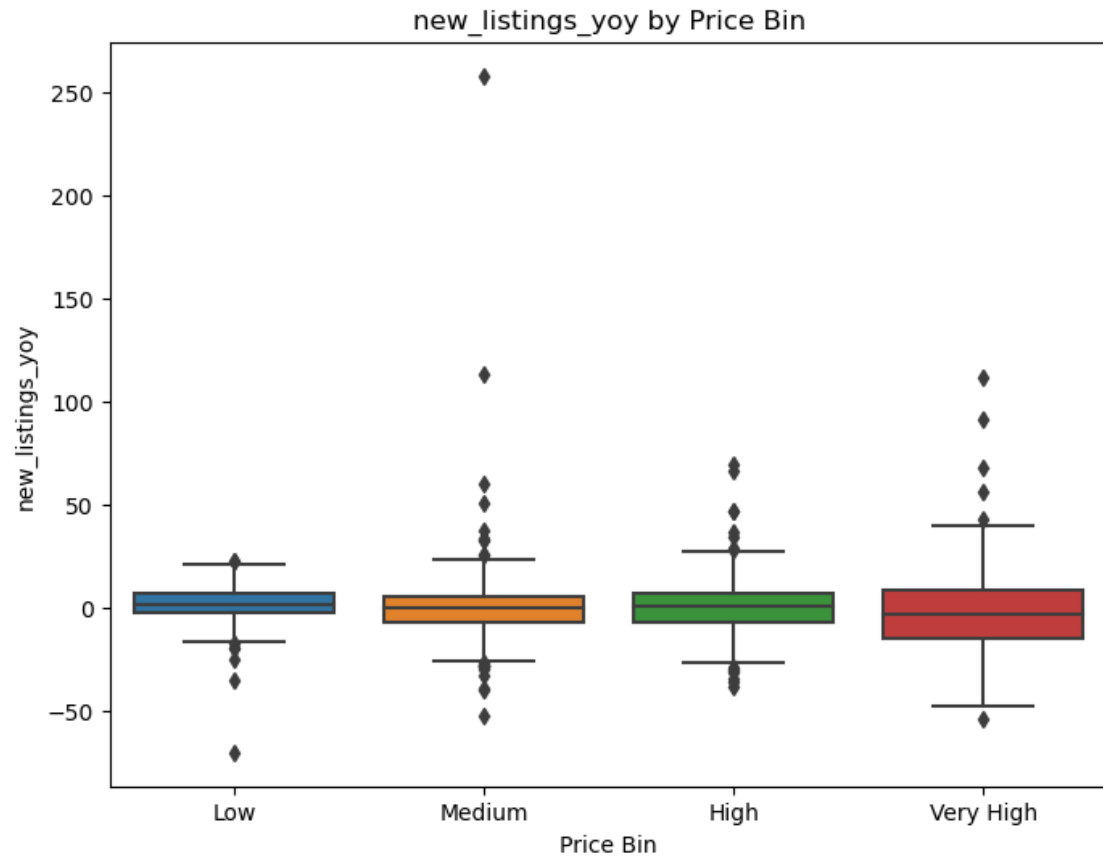


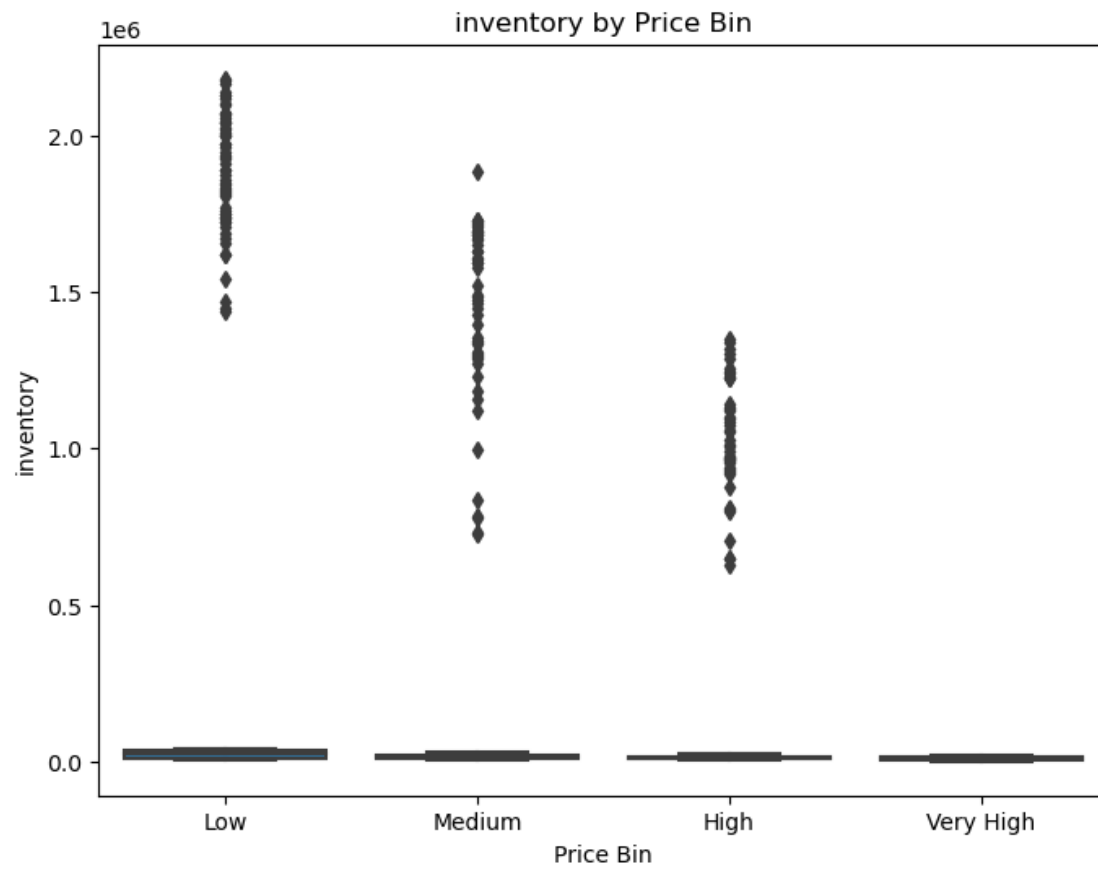


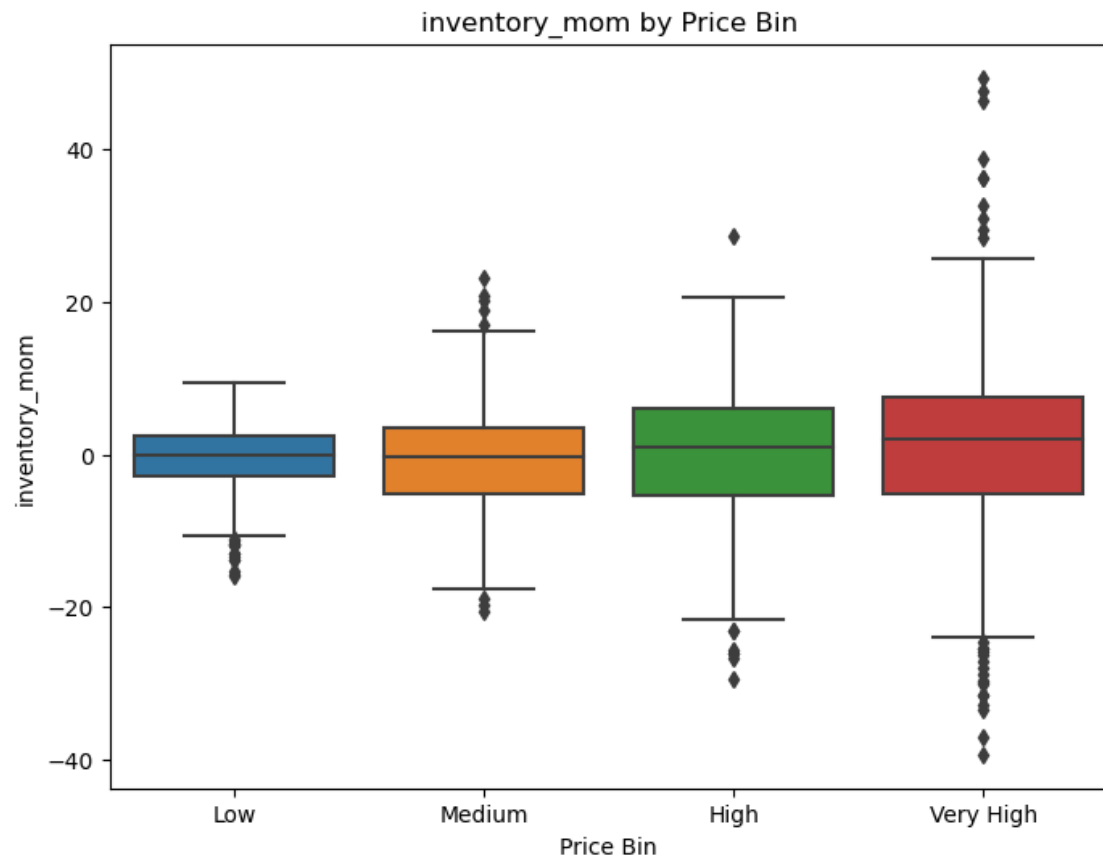


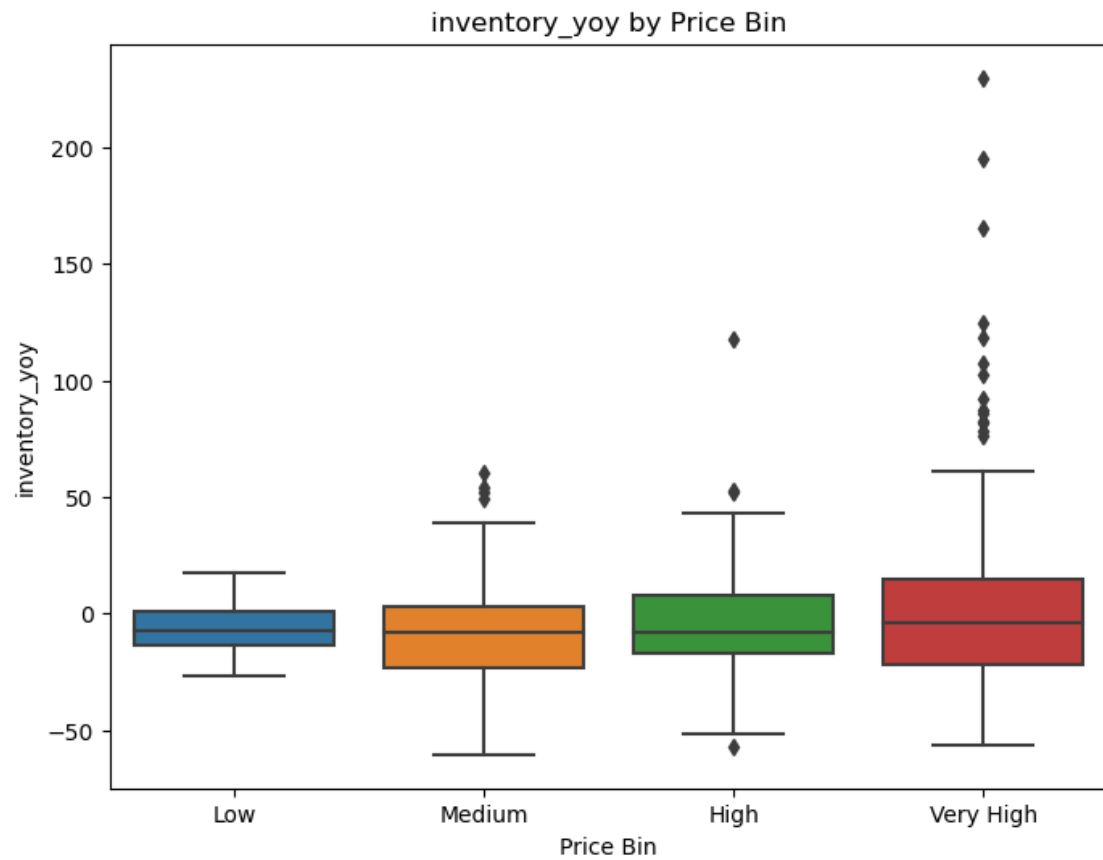


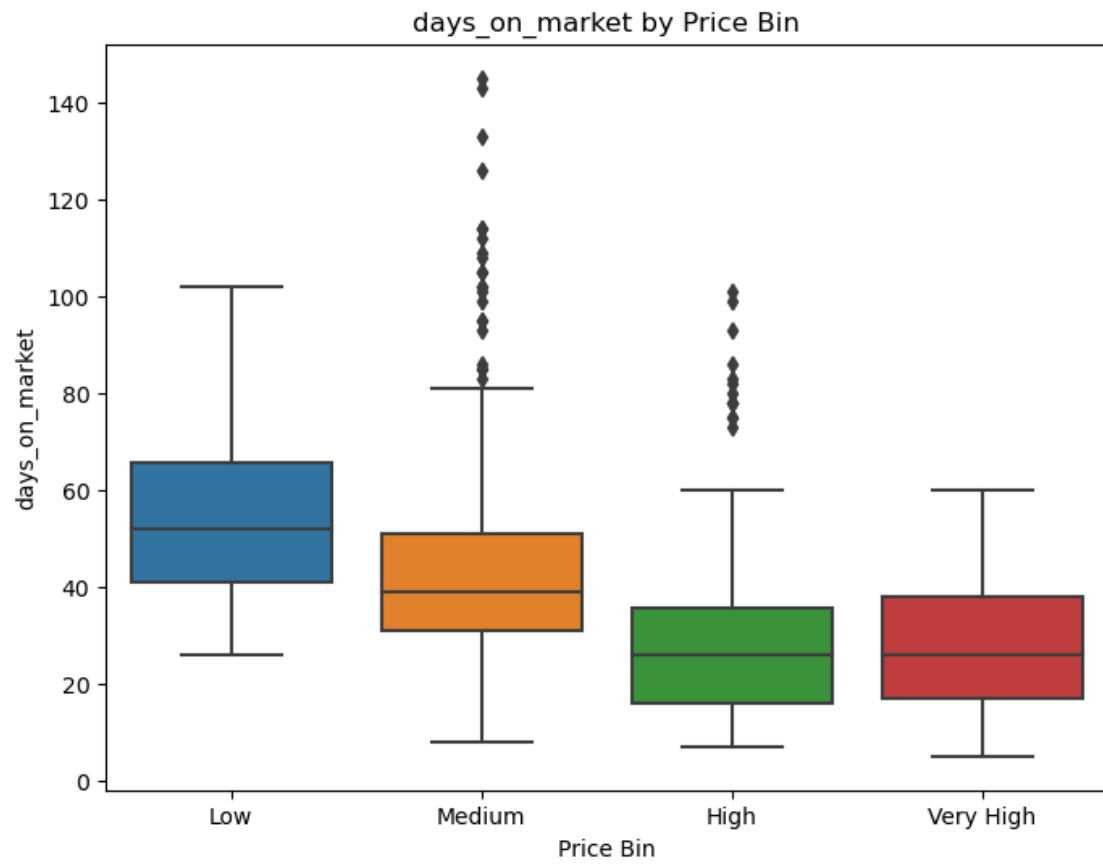


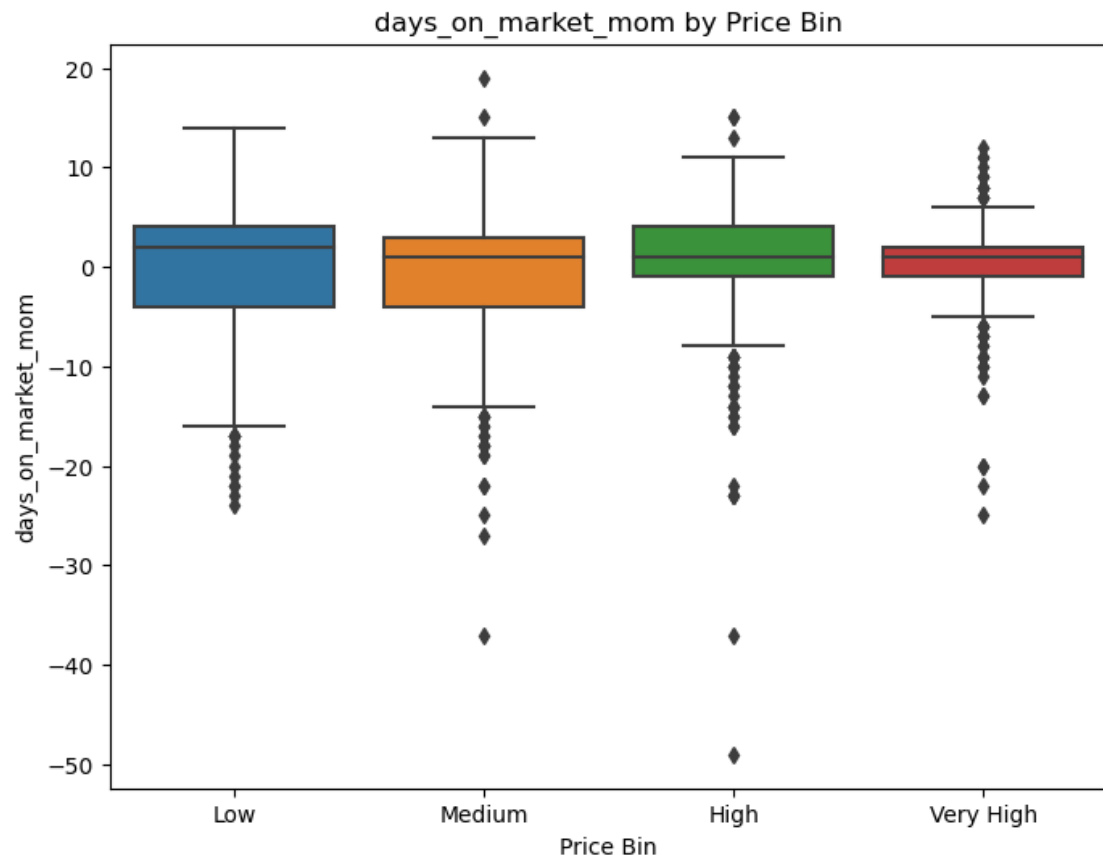


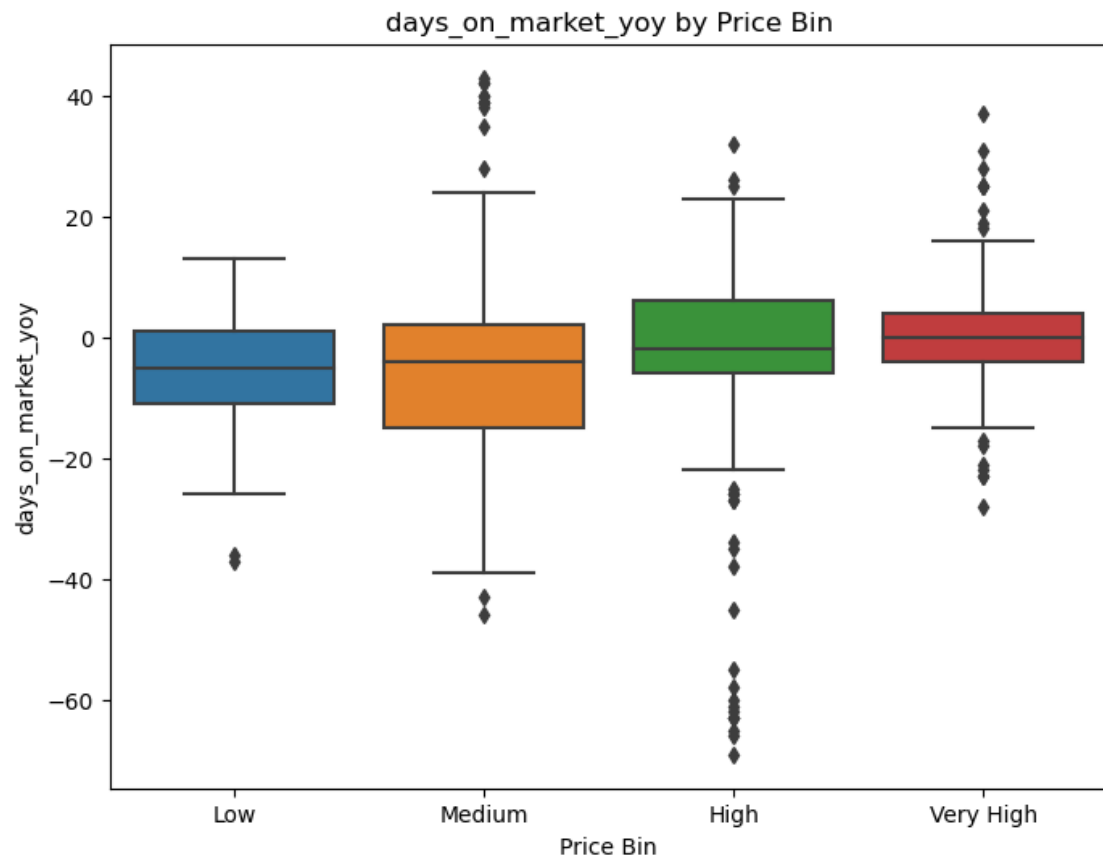


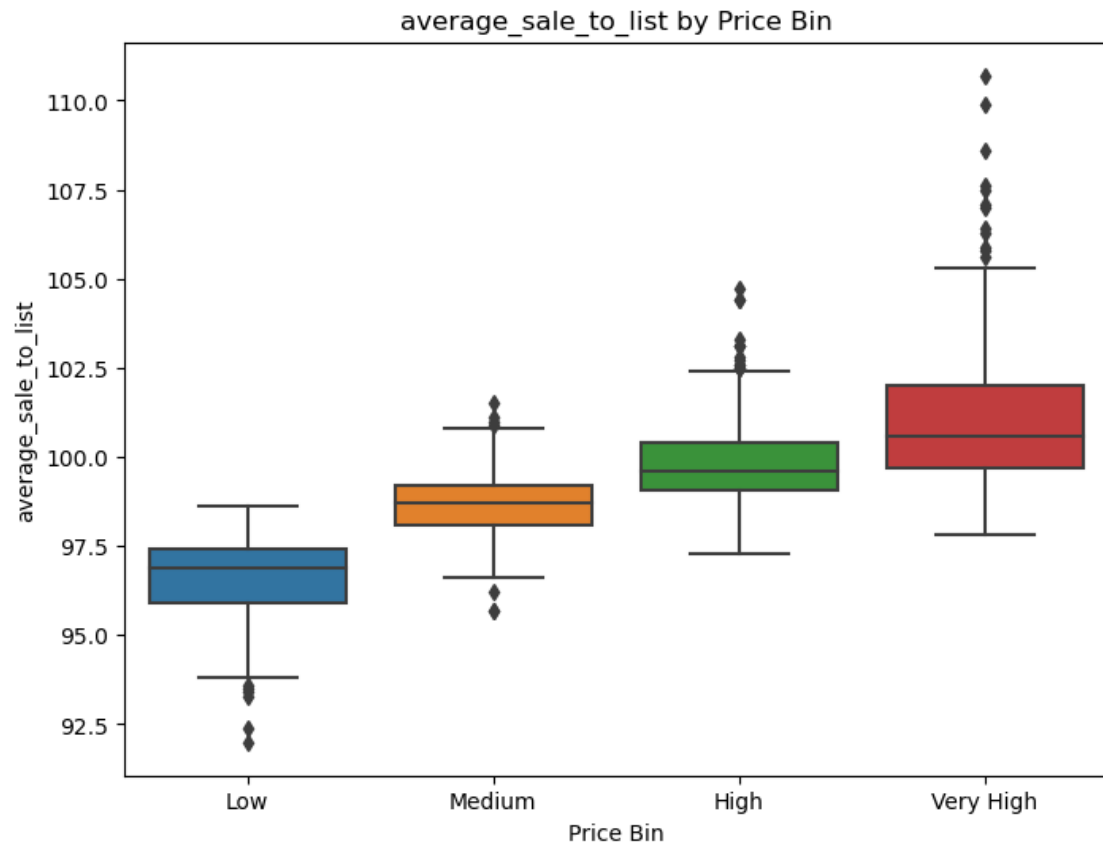


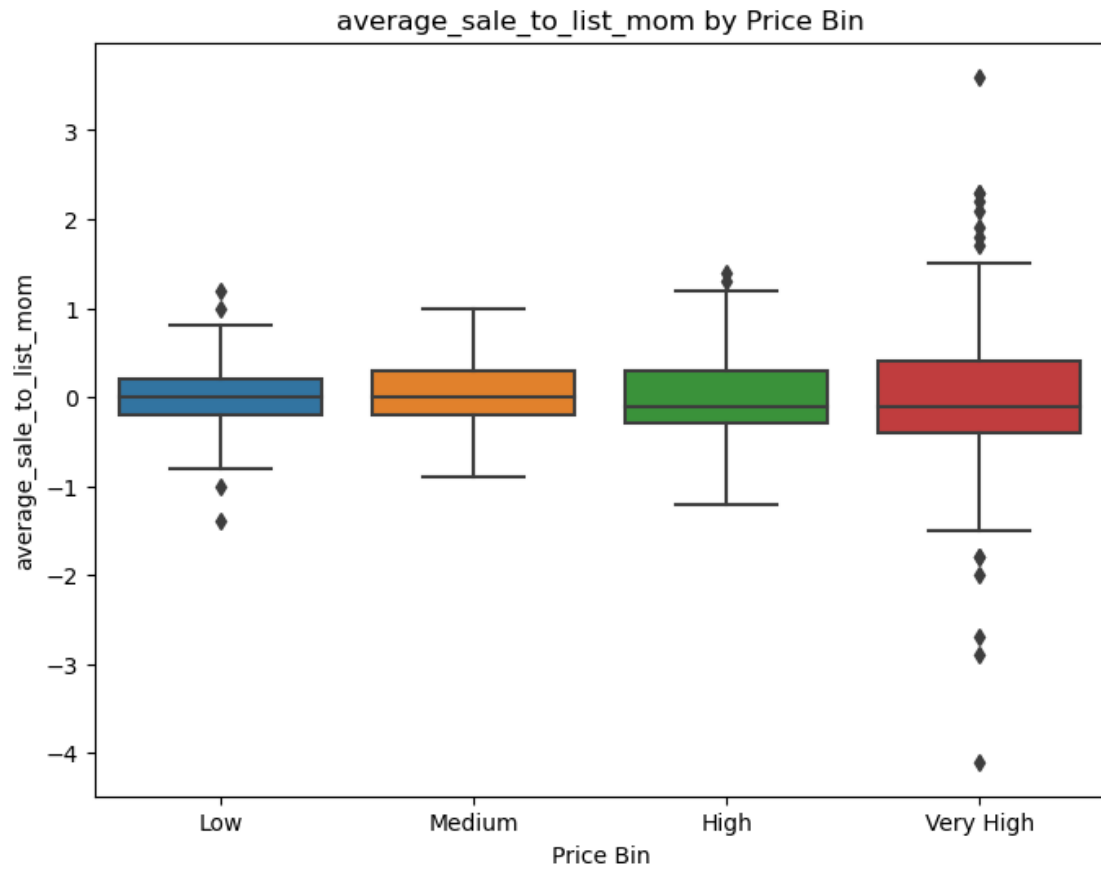


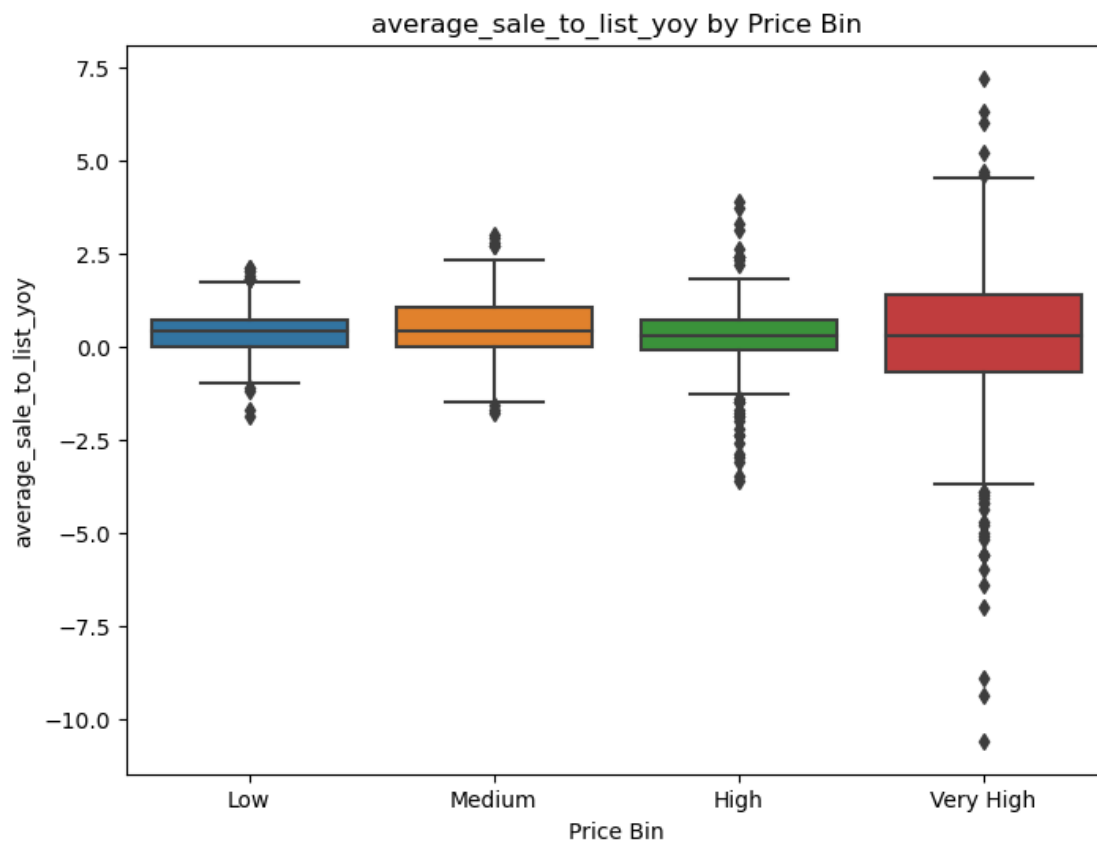


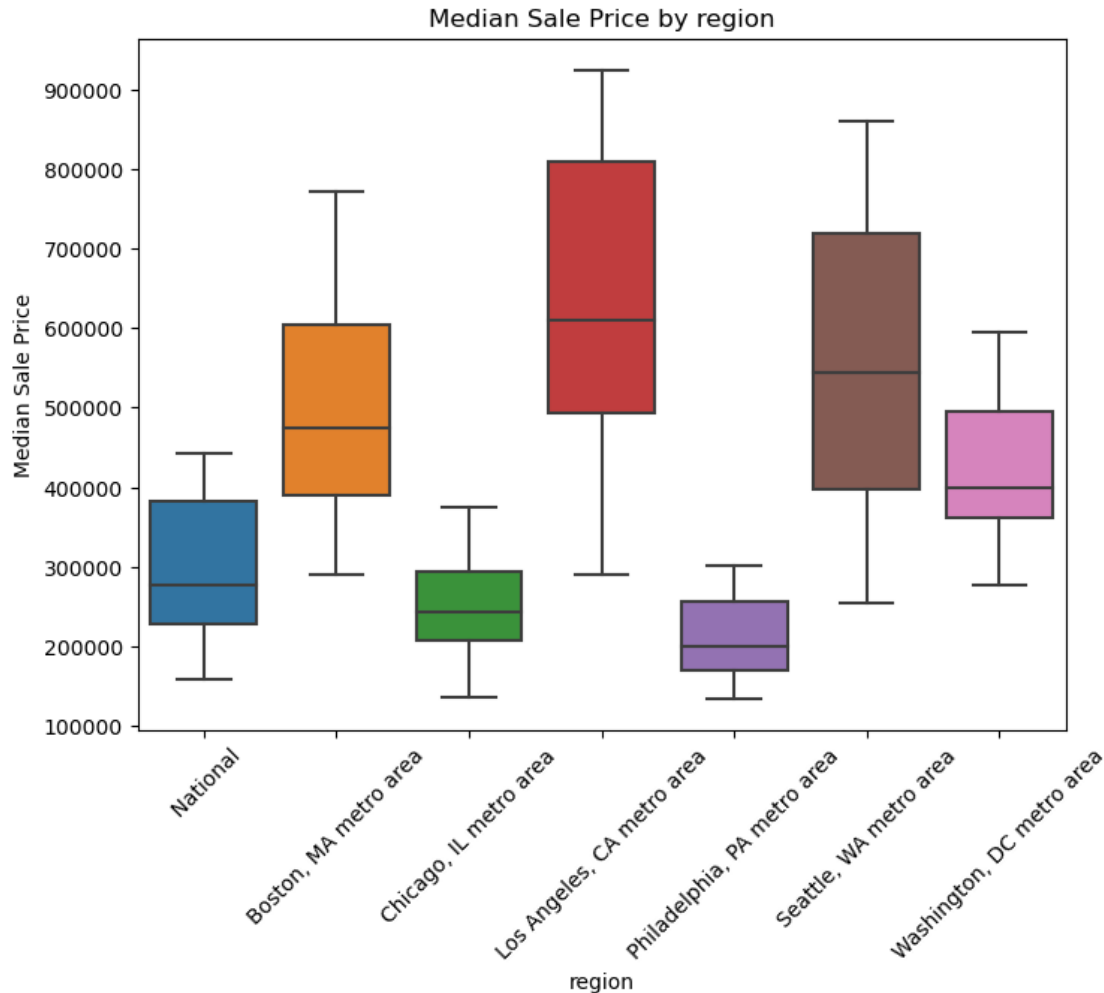












BoxPlot Insights

Below is an explanation of the key boxplots I've produced and why they matter for my project. Rather than listing every single variable, I'm focusing on those with notable outliers or high relevance to house-price modeling.

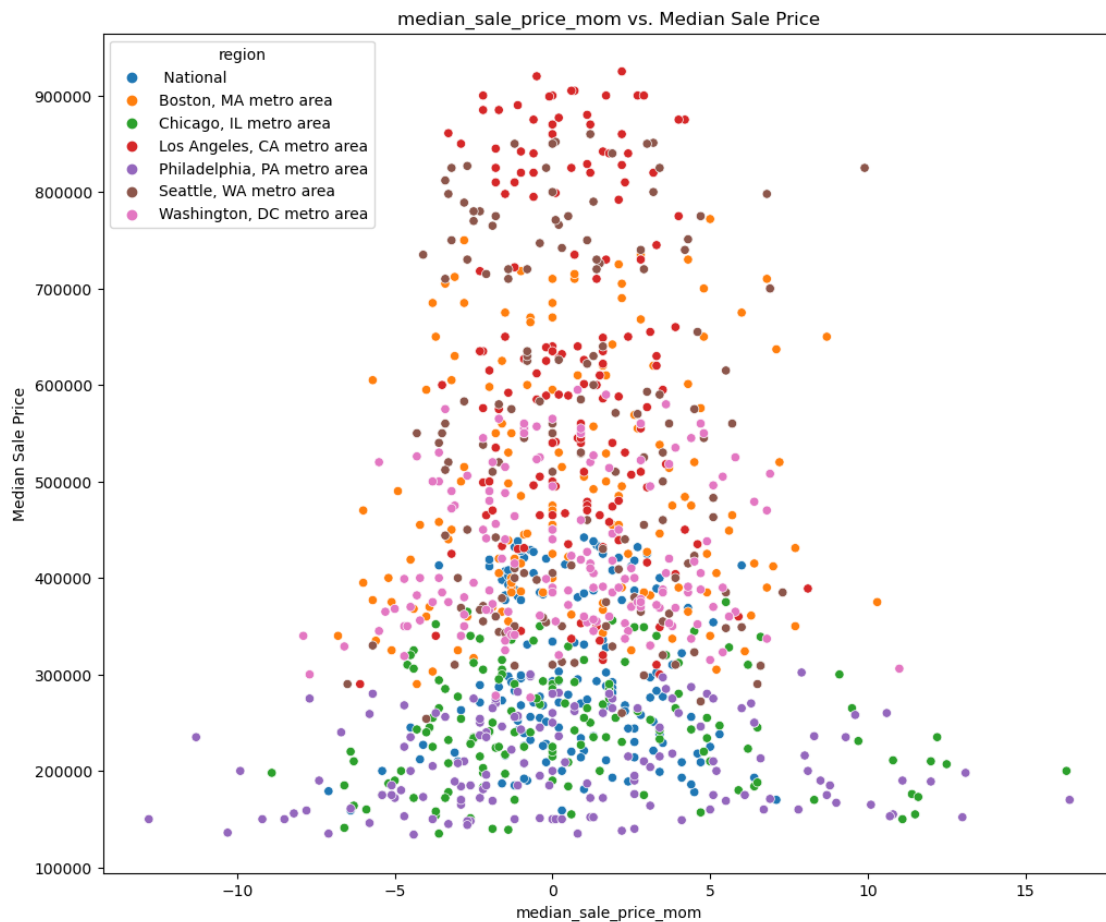
- Homes Sold (Boxplot) - Notable Outliers:** A few points stand far above the upper whisker, indicating especially high sales in certain months/regions. - **Relevance:** Large spikes in homes sold can influence average prices and reflect heightened demand. Such outliers may indicate unusually hot markets or seasonal booms (e.g., spring).
- Days on Market (Boxplot) - Outliers:** Some properties linger on the market significantly longer than typical, which can extend well beyond 100+ days. - **Relevance:** Days on market is inversely correlated with price—when houses sell quickly, prices are often higher. The outliers could represent unique property types, overpriced listings, or less desirable locations.
- Inventory (Boxplot) - Outliers:** A handful of months/regions show extremely high inventory levels, well above typical ranges. - **Relevance:** Excessive inventory can apply downward pressure on prices. Outlier months might coincide with shifts in economic conditions or new construction spikes.
- Median Sale Price (Boxplot) - Skew & High-Price Outliers:** The boxplot is right-skewed; some luxury or high-demand areas push prices well above the median. - **Relevance:** These outliers are integral to your analysis, because price is

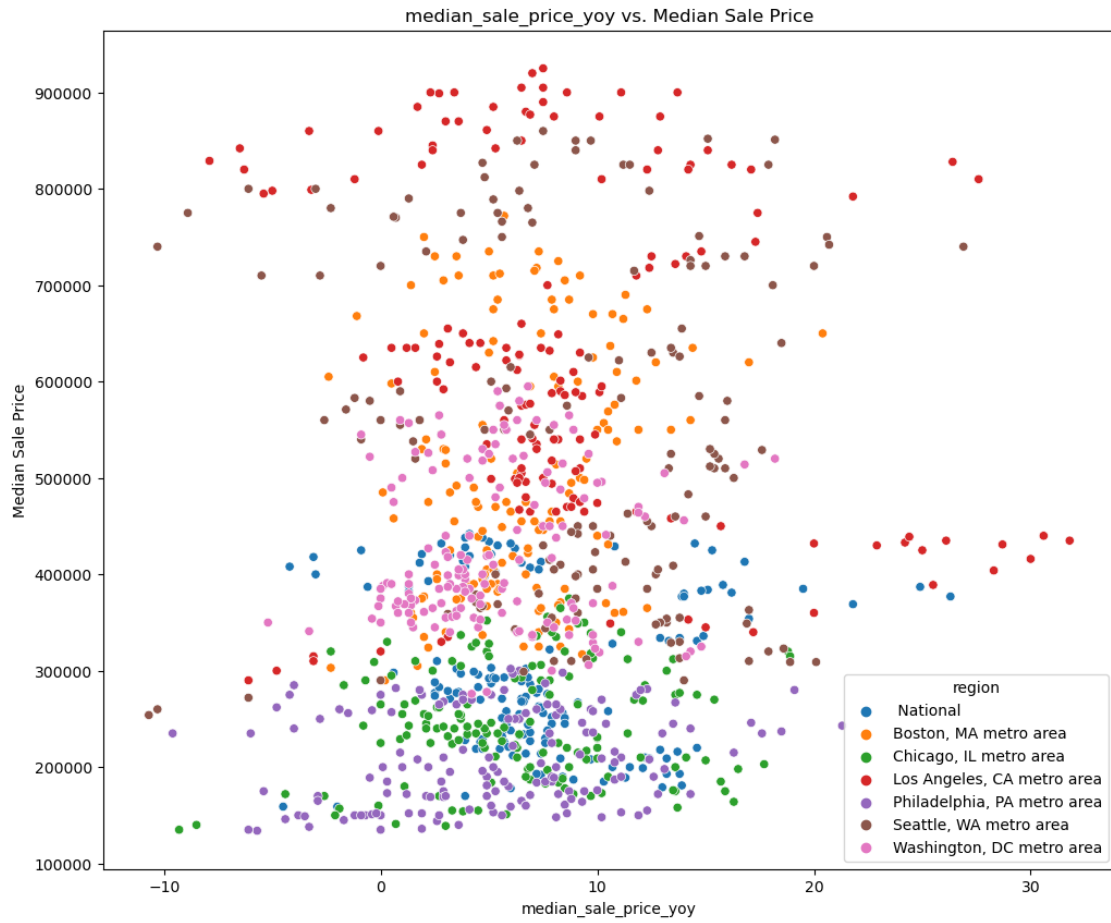
the target variable. Log transformation or robust regressions might help handle these extremes in modeling. 5. Boxplot by price_bin - Why Binning?: Converting median_sale_price to categories (e.g., Low, Medium, High) allows us to see how other metrics (like homes_sold, days_on_market) differ across price tiers. - Key Observations: Higher-tier price bins often correlate with fewer days on market and slightly lower inventory, suggesting strong demand in more expensive markets. A small subset of extreme outliers can appear in each bin (e.g., unusually fast or slow sales).

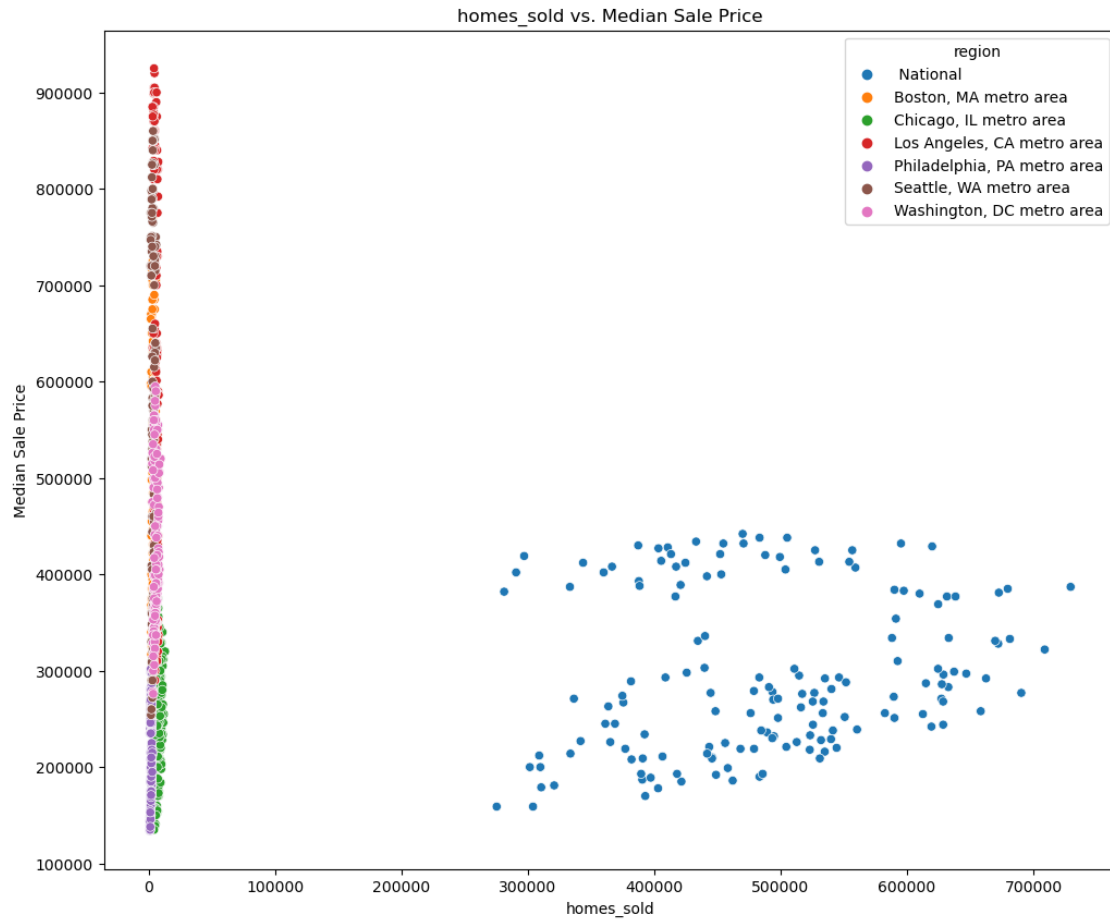
Next Steps

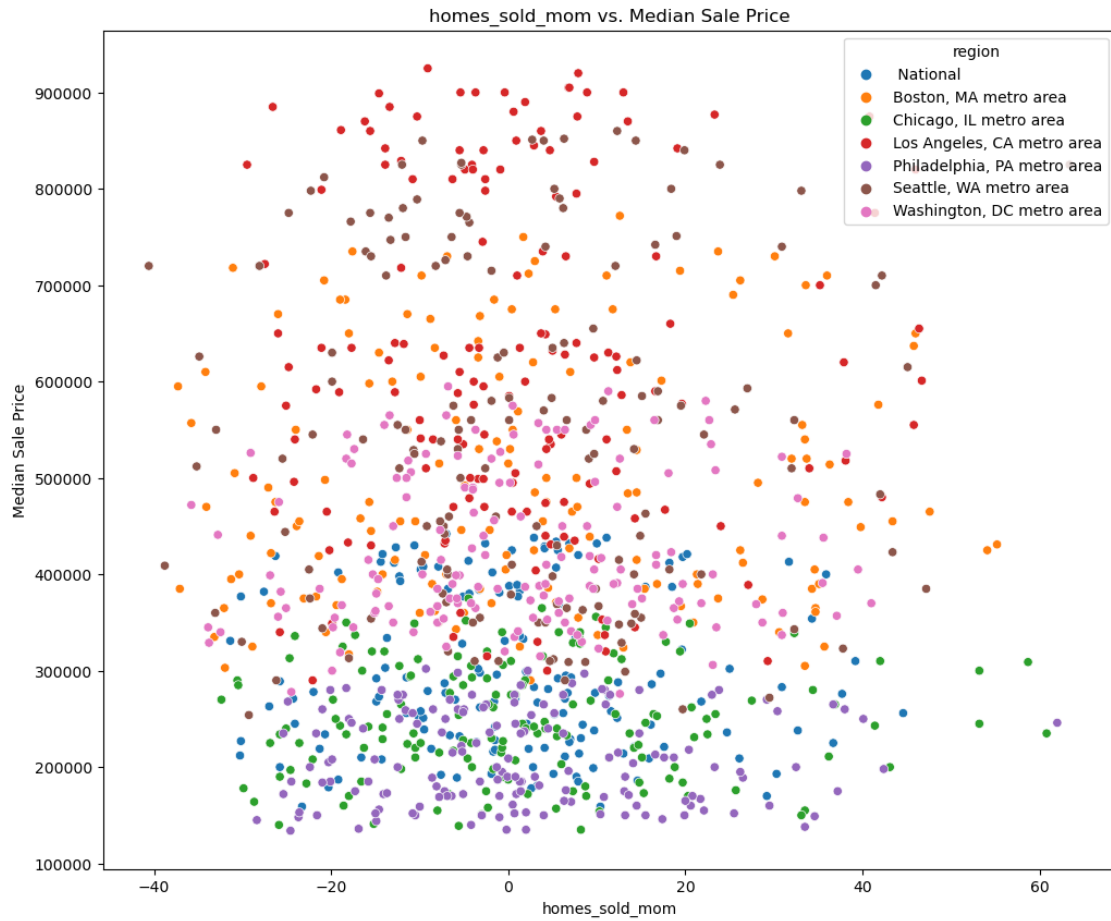
- Decide on Outlier Treatment: For modeling, should I exclude or transform extreme values?
- Connect Boxplots to Time-Series: Check whether outlier months line up with anomalies in my seasonal decomposition or other metrics (e.g., interest rate changes).
- Refine Feature Selection: Outliers can distort correlations. I will consider robust scaling or transformations (log, sqrt) to reduce skew.

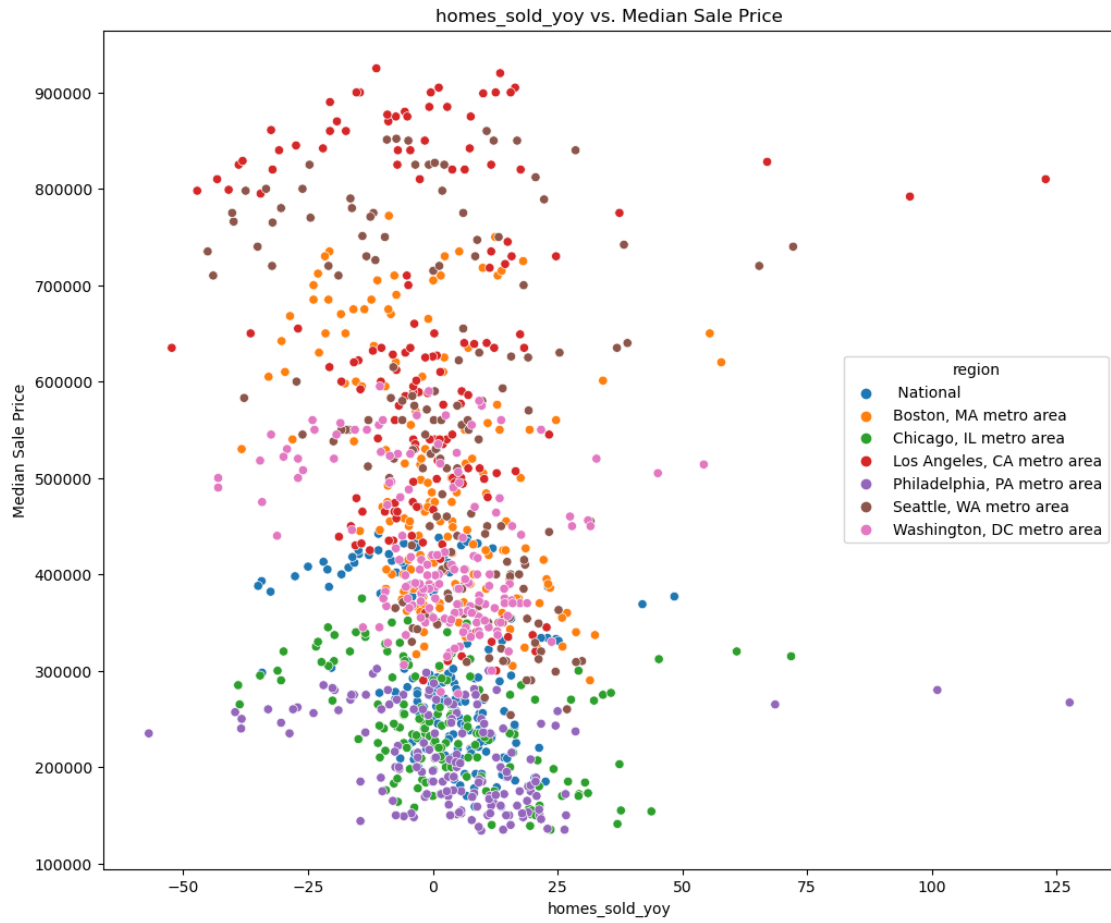
```
[12]: e.bivariate_scatterplot(redfin_df, numerical_cols, categorical_cols)
```

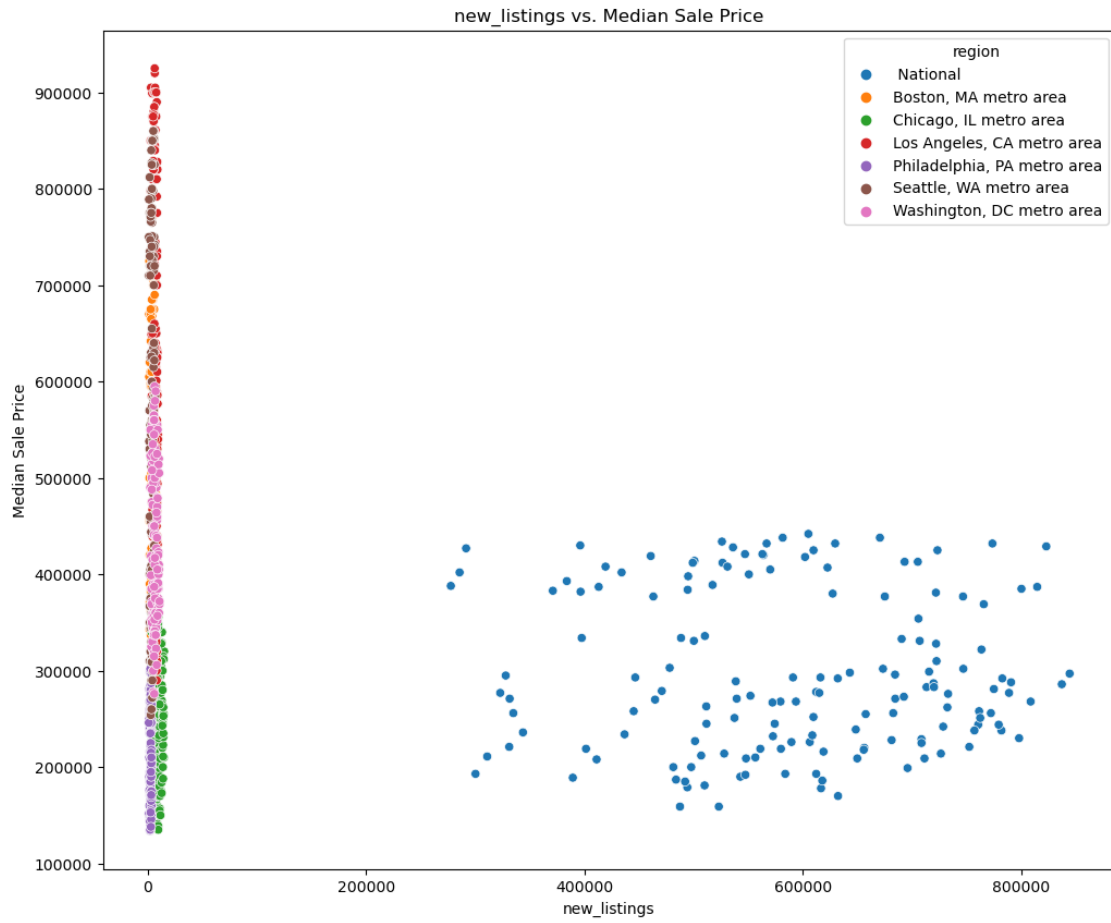


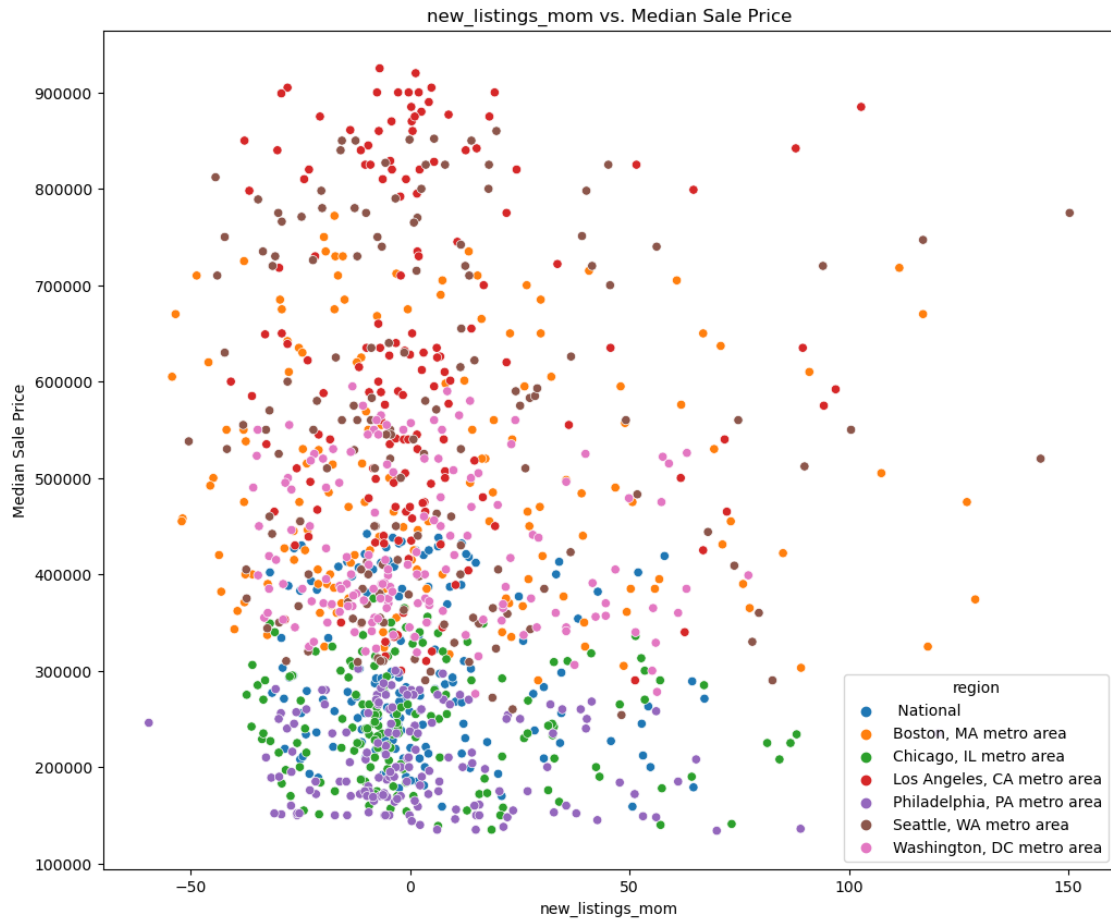


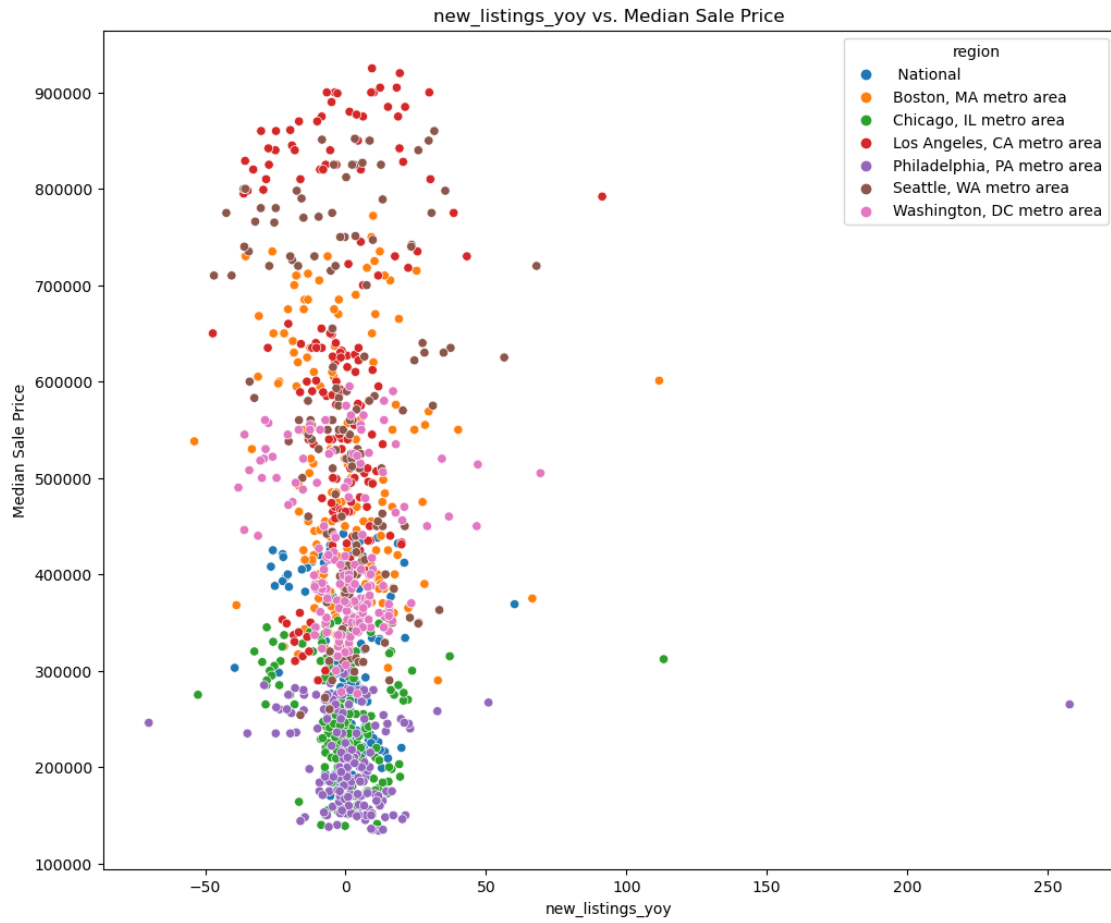


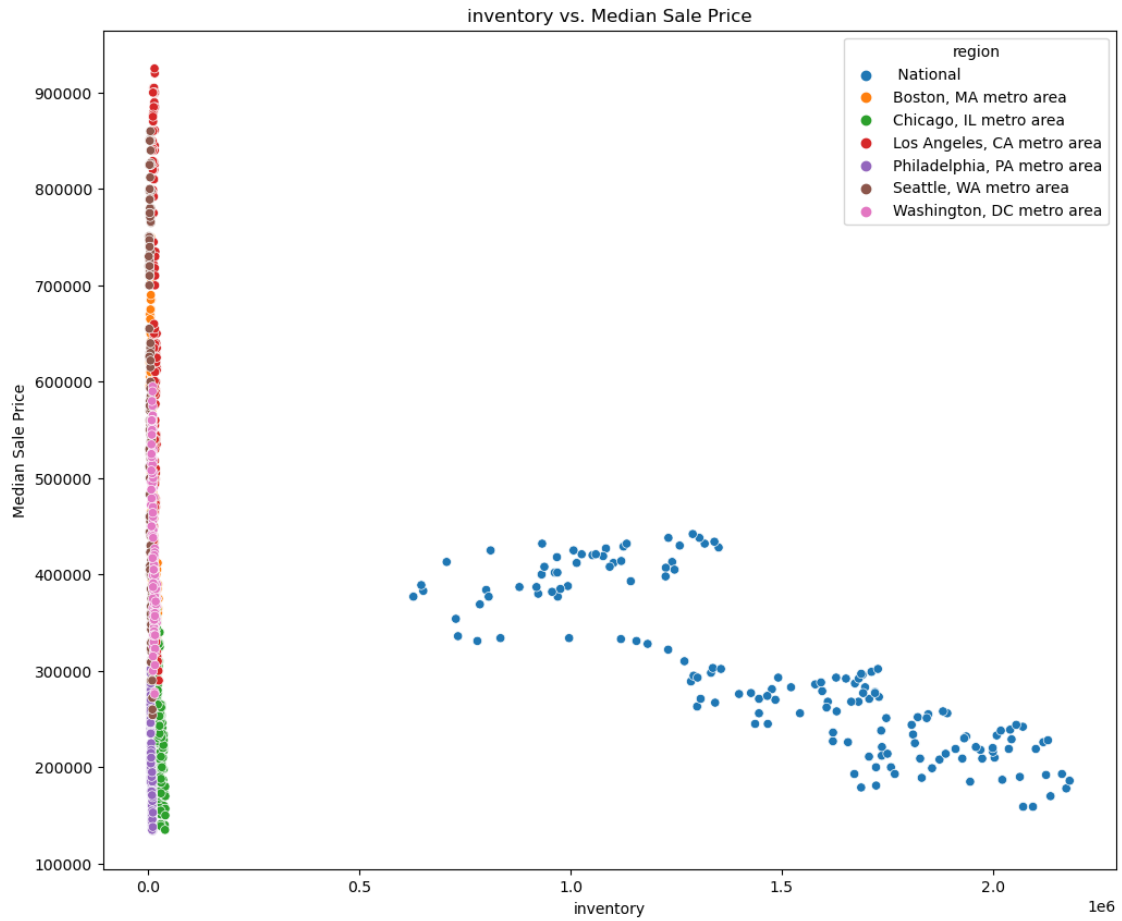


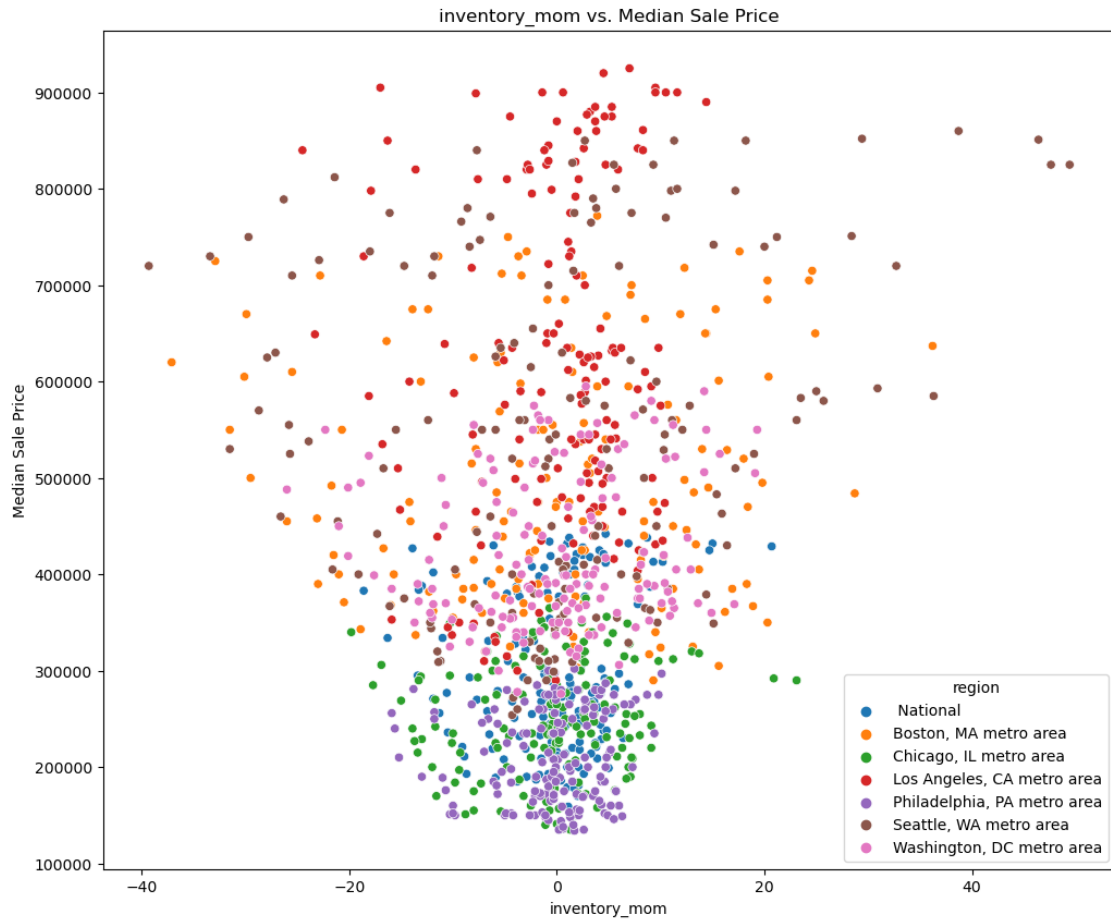




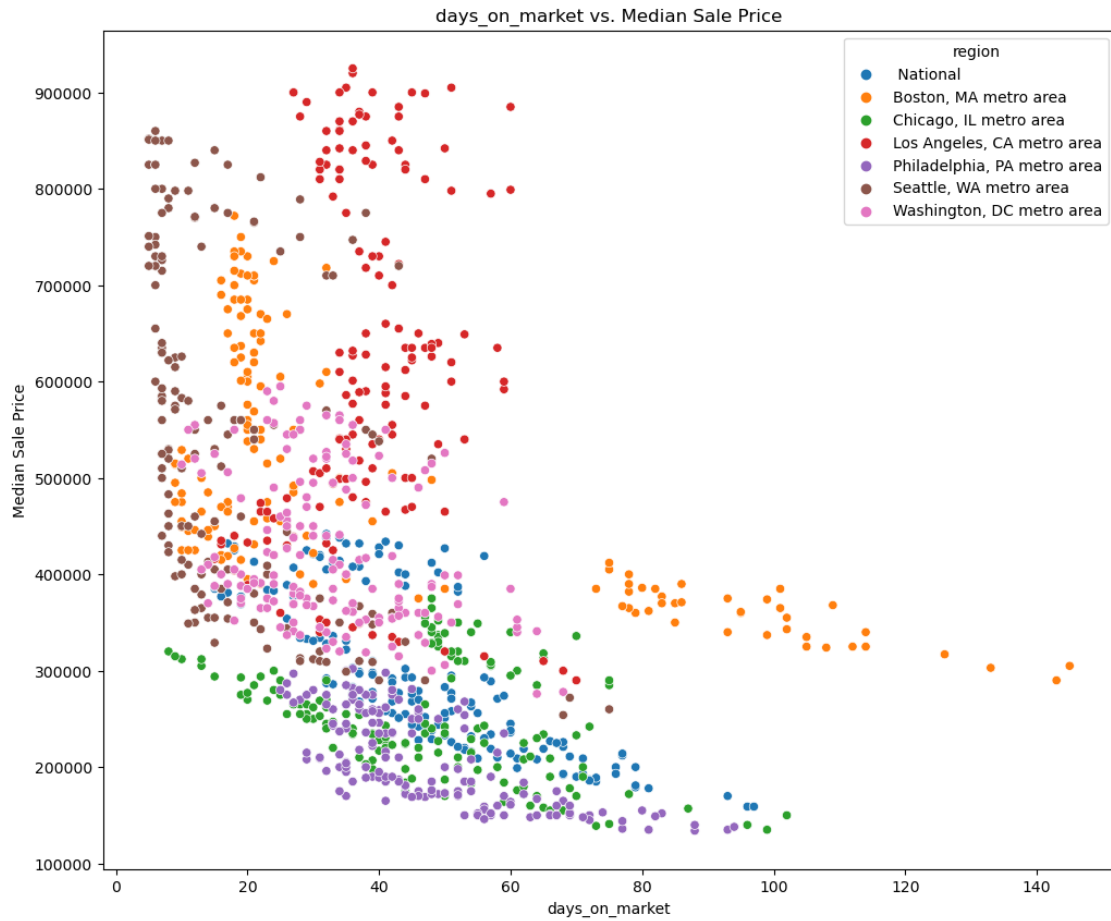


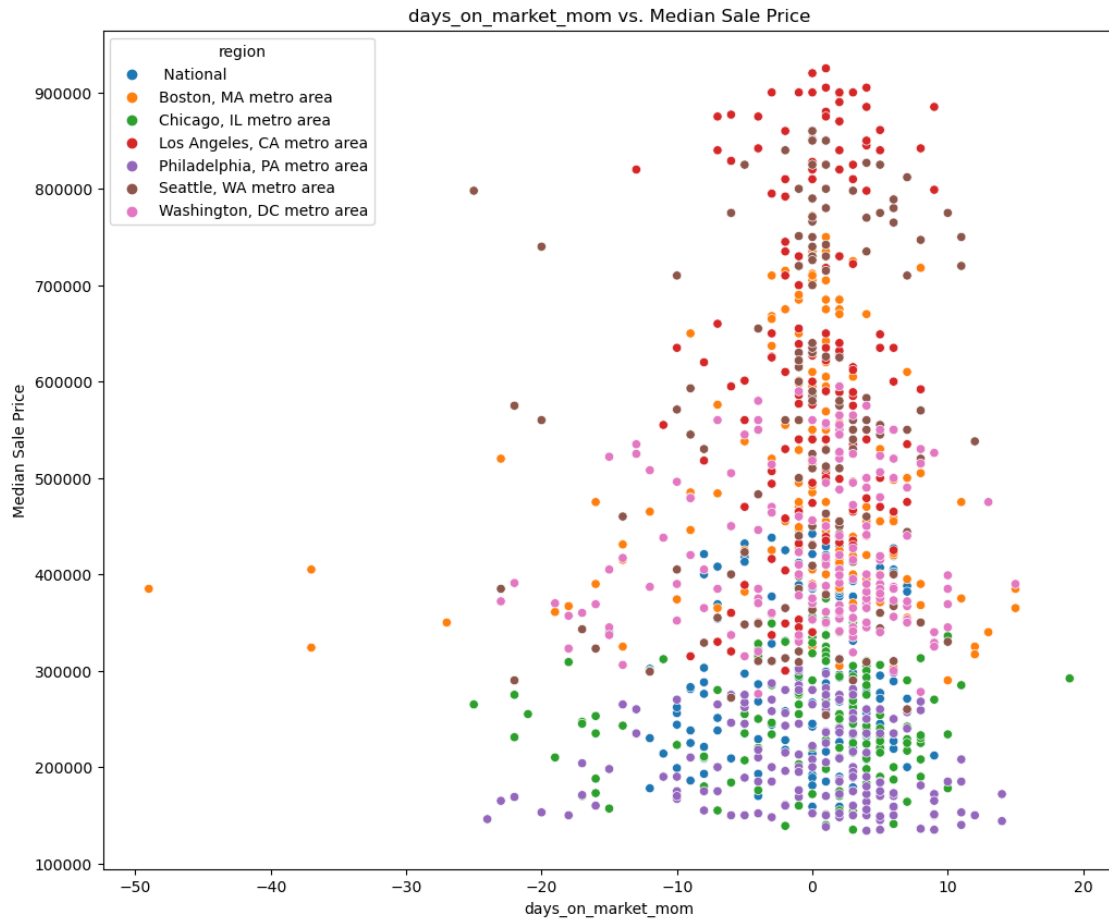


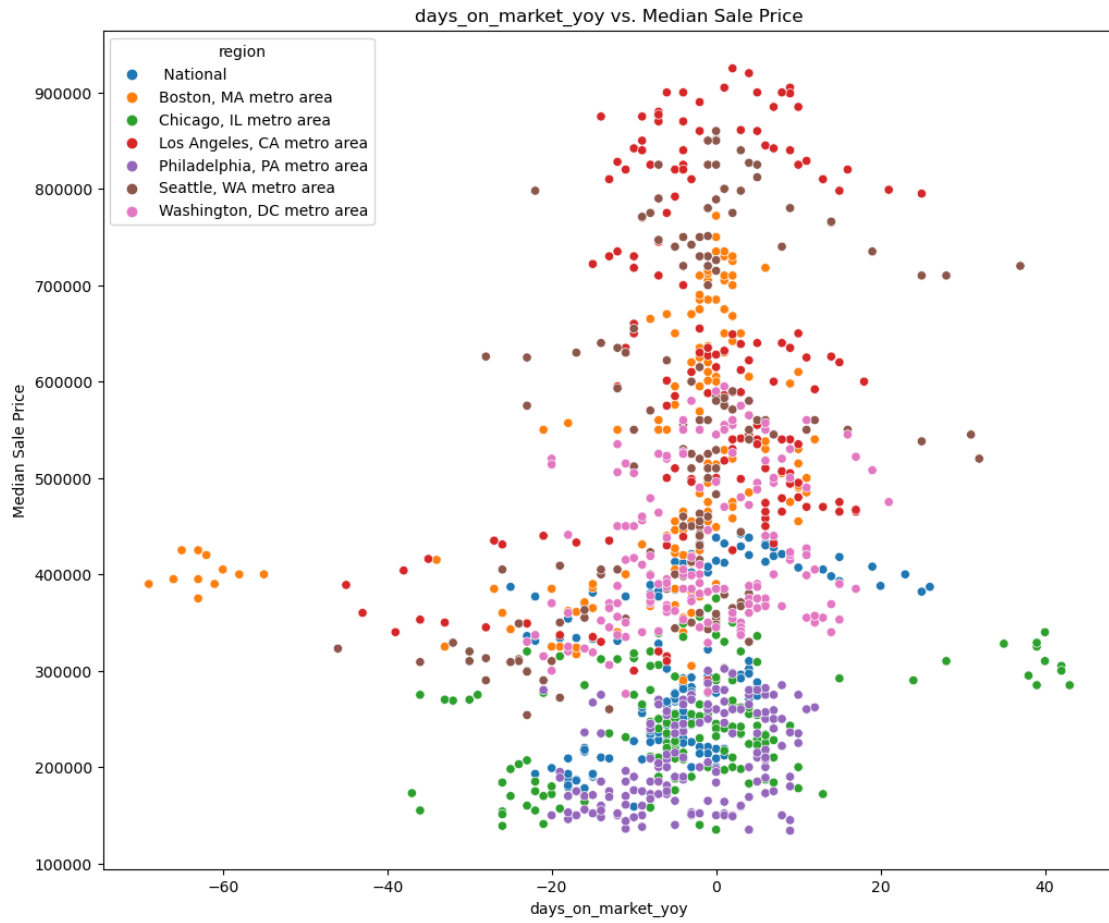


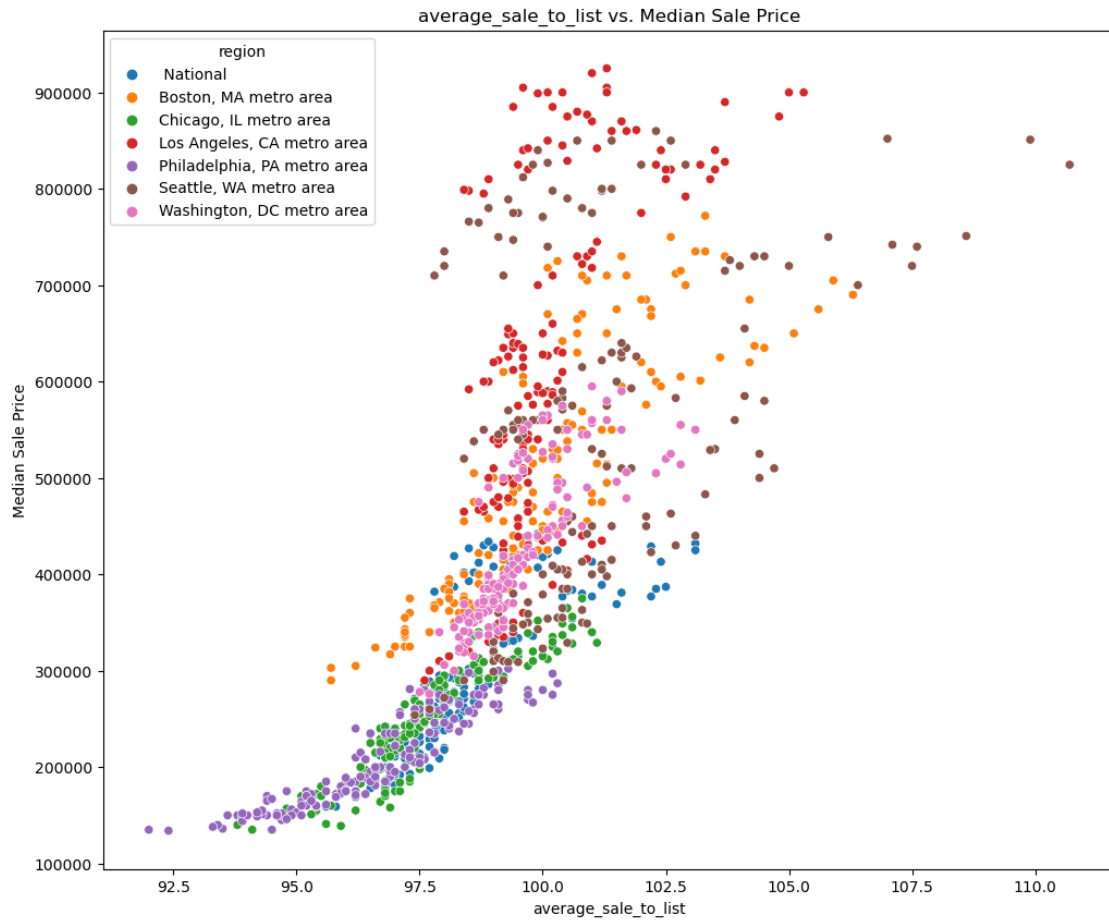


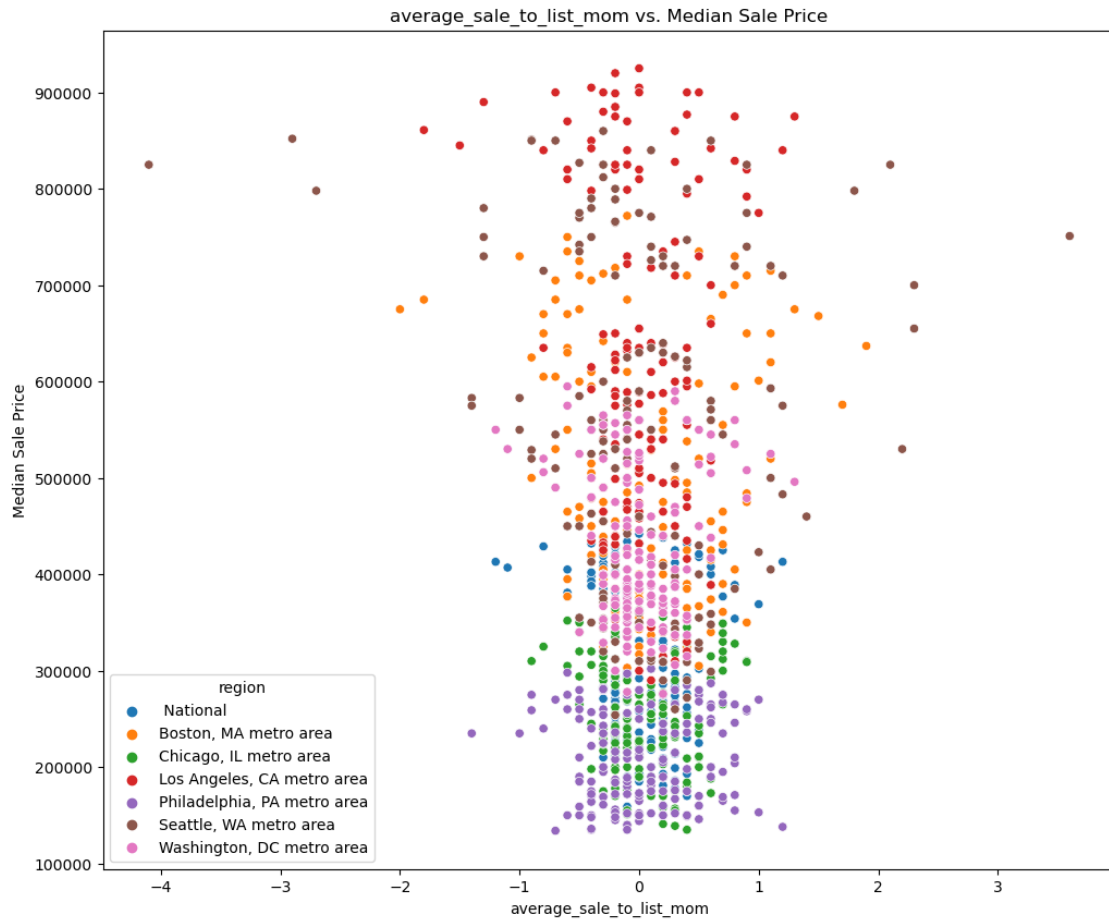


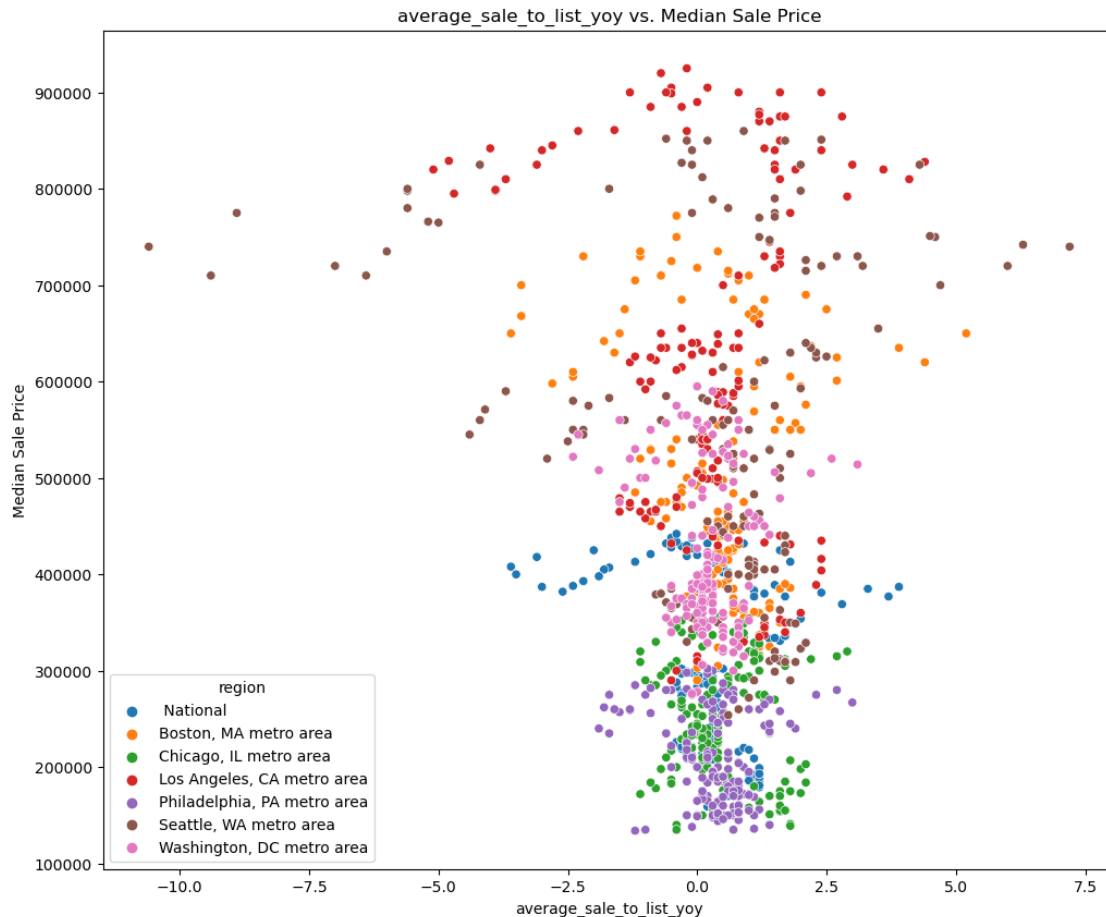












ScatterPlot Insights

Below is an interpretation of the scatterplots I've generated.

1. Overarching Themes & Patterns

1. Region-Specific Clustering - Boston (Orange) repeatedly stands out for higher median prices (~\$550k–\$600k) with short days on market (under ~25 days), suggesting a hot market with sustained demand. - Chicago (Green) shows moderate prices (~\$300k–\$400k) but occasional spikes above 60 days on market, implying seasonal or economic slowdowns. - Los Angeles (Red) often splits into two sub-markets: quick sales at moderate-to-high price levels and pricier listings lingering above 50 days on market, reflecting LA's diverse property segments. - Seattle (Brown) experiences inventory surges that don't immediately drag down prices—an early sign that, if demand weakens, a more noticeable price dip could follow.
2. Inventory vs. Price Lags - Across multiple scatterplots (especially for Seattle and the National data in Blue), we see rising inventory levels without an immediate drop in median sale prices. - This lag effect suggests that supply growth takes time to influence final sale prices—aligning with the idea that inventory can be an early warning measure.
3. Sales Volumes & Days on Market - Washington, DC (Pink) and Philadelphia (Purple) generally hover in a midrange of 20–40 days on market, with moderate price points. However, DC shows a relatively high volume of sales, indicating quick condo turnover or robust buyer interest. - When days on market does spike, the scatterplots show it often corresponds to slower sales volume in

certain months, clarifying earlier boxplot outliers as tied to specific time frames or local conditions.

— 2. New Insights Beyond the Boxplots

1. Identifying Hidden Anomalies - Several scatterplots reveal “micro outliers” that don’t appear extreme enough in boxplots to be flagged, but stand out when comparing two variables (e.g., extremely fast sales in a mid-price bracket for a specific region or month). - These subtle anomalies can signal pockets of unique activity, such as sudden investor interest or localized job growth, invisible in a univariate boxplot. 2. Leading Indicators for Market Shifts - In LA, DC, and Seattle especially, we see how an uptick in days on market or a surge in inventory often foreshadows a slight flattening or gradual decrease in prices a few months later. - This suggests a predictive dimension: if these patterns are captured in a time-series or machine learning model, they might serve as early warning signs for price shifts. 3. Region vs. National Discrepancies - The National (Blue) plots smooth out local spikes, confirming that aggregated data can mask volatile patterns visible only when looking at individual metros. - This underscores the importance of regional modeling: markets like Boston or LA behave differently than the national average, which could improve the hypothesis about analyzing local supply-demand factors. — 3. Tying It All to the Hypothesis

Hypothesis Statement: Housing market trends can be predicted by analyzing historical price fluctuations, inventory changes, and sales patterns. By applying machine learning techniques, we can identify early indicators of market shifts, allowing for better-informed real estate investment decisions.

- The scatterplots confirm that:
 1. Price, inventory, and days on market each show distinct patterns by region.
 2. Lag effects (inventory changes preceding price moves) frequently appear, signifying an early signal capability.
 3. Outliers and sub-market clusters identified here reinforce the need to incorporate region-specific characteristics into any predictive model.

With these visual confirmations, the data strongly supports the hypothesis that local inventory spikes, fluctuating days on market, and changes in sales volume can foretell upcoming price adjustments. A machine learning approach integrating these features—especially broken down by metro area—stands a good chance of capturing and predicting market shifts more accurately than a one-size-fits-all national model.

```
[13]: redfin_df.groupby('region')['price_bin'].describe()
```

```
[13]:
```

	count	unique	top	freq
region				
National	157	3	Low	61
Boston, MA metro area	157	3	High	71
Chicago, IL metro area	157	3	Low	94
Los Angeles, CA metro area	157	3	Very High	106
Philadelphia, PA metro area	157	2	Low	119
Seattle, WA metro area	157	4	Very High	85
Washington, DC metro area	157	3	High	83

Price_bin distribution by region Insights

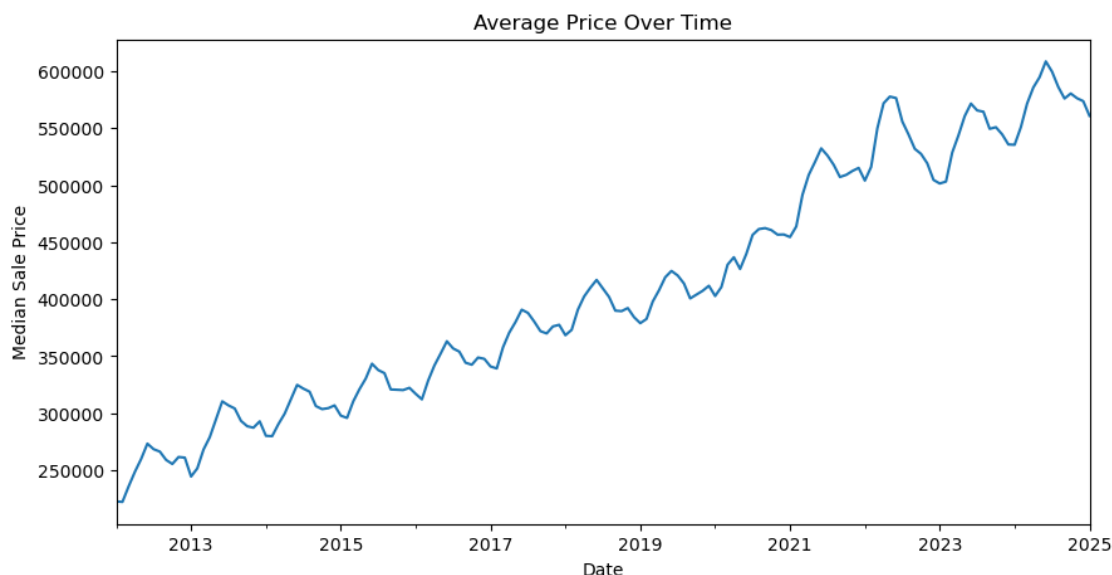
Below is overview of the price_bin distribution by region. Each region has 157 rows of data, and

the table shows which bin is most frequent (“top”) as well as how many times it appears (“freq”):

- Philadelphia and Chicago remain primarily “Low” price bins, suggesting generally lower median prices compared to the other metros.
- Boston and DC hover around the “High” bin most often, indicating a consistently elevated market.
- LA and Seattle both stand out with “Very High” as their predominant bin, showcasing top-tier market prices.
- National data is skewed to “Low”, which makes sense as a broad average across diverse regions—including many less expensive areas beyond major metro hubs.

Overall, these frequency counts confirm each metro’s price tier in the dataset, supporting the broader trends observed in the scatterplots and boxplots (e.g., LA’s and Seattle’s especially high prices, Philadelphia’s lower cost, etc.).

```
[14]: e.plot_average_price_over_time(redfin_df)
```



Average Price Over Time Insights

Below is an interpretation of the “Average Price Over Time” line chart and how it confirms or extends what we’ve seen in the other plots.

1. Overall Upward Trajectory

From about \$250k in 2012 to roughly \$550k+ by late 2024, the chart depicts a steady, long-term increase in median sale prices at the national level.

- This upward climb aligns with the Nationwide “Low” bin we saw in price_bin distributions; although it’s labeled “Low” relative to high-cost metros like Los Angeles or Seattle, the absolute trend is still consistently rising.

•

3.1 It also matches our time-series decomposition results (and scatterplots) indicating that price dips were mostly short-lived despite fluctuations in inventory or days on market.

2. Mid-2019 to 2021 Acceleration Around 2019, the slope appears to steepen briefly, followed by a small plateau in 2023–2024.
 - Some earlier scatterplots (e.g., `inventory_mom` vs. `median_sale_price`) showed that surges in inventory didn’t immediately reduce prices—this line chart confirms a continued upward price movement during that period.
 -

3.2 The slight plateau in 2023–2024 could tie back to the occasional inventory spikes or days-on-market increases we saw for certain metros, suggesting a softening demand or a leveling out in the national average.

3. Tying Back to Region-Level Diversity

While this chart aggregates the National data, we know from the region-based boxplots and scatterplots that individual metros like Boston or Los Angeles often exceed these values, whereas places like Chicago or Philadelphia consistently fall below.

-

3.3 The overall national upward line masks the substantial variations visible in local scatterplots. L.A. (Red) or Seattle (Brown) can be “Very High,” while Philly (Purple) is mostly “Low,” yet everyone is trending upward.

Final Takeaway

This time-series line confirms a broad, long-run price appreciation, reinforcing that local anomalies (whether spikes in inventory or changes in days on market) have not derailed the national climb. It fits well with the earlier findings that supply surges sometimes lag before affecting prices and that regional differences are key, even within an overall upward trend.

3.3.1 Week 3 EDA

Week 3: - Tasks: - Perform feature engineering. - Handle missing values and outliers. - Goal: Prepare a well-structured dataset for modeling.

Outliers Why Are We Checking for Outliers?

The goal of this step is to **identify and handle extreme values** that could distort our analysis and model performance. Outliers can arise from **data entry errors, rare events, or genuine market fluctuations**, and it’s crucial to assess their impact.

- **Why It Matters:**
 - Outliers can **skew statistical summaries** (e.g., mean, standard deviation).

- They may **negatively impact model training**, especially in regression and machine learning algorithms.
- Identifying them helps decide whether to **remove, transform, or cap** extreme values for better data integrity.

By detecting outliers, we ensure that our dataset remains **accurate, reliable, and representative** of real-world trends.

```
[15]: # Checking for Outliers
outliers = e.detect_outliers_iqr(redfin_df)
print(outliers)
```

```
[15]: median_sale_price      0
      median_sale_price_mom  27
      median_sale_price_yoy  37
      homes_sold            157
      homes_sold_mom        17
      homes_sold_yoy         62
      new_listings          157
      new_listings_mom       79
      new_listings_yoy       88
      inventory             157
      inventory_mom          84
      inventory_yoy          46
      days_on_market         29
      days_on_market_mom     80
      days_on_market_yoy     75
      average_sale_to_list    41
      average_sale_to_list_mom 50
      average_sale_to_list_yoy 97
      dtype: int64
```

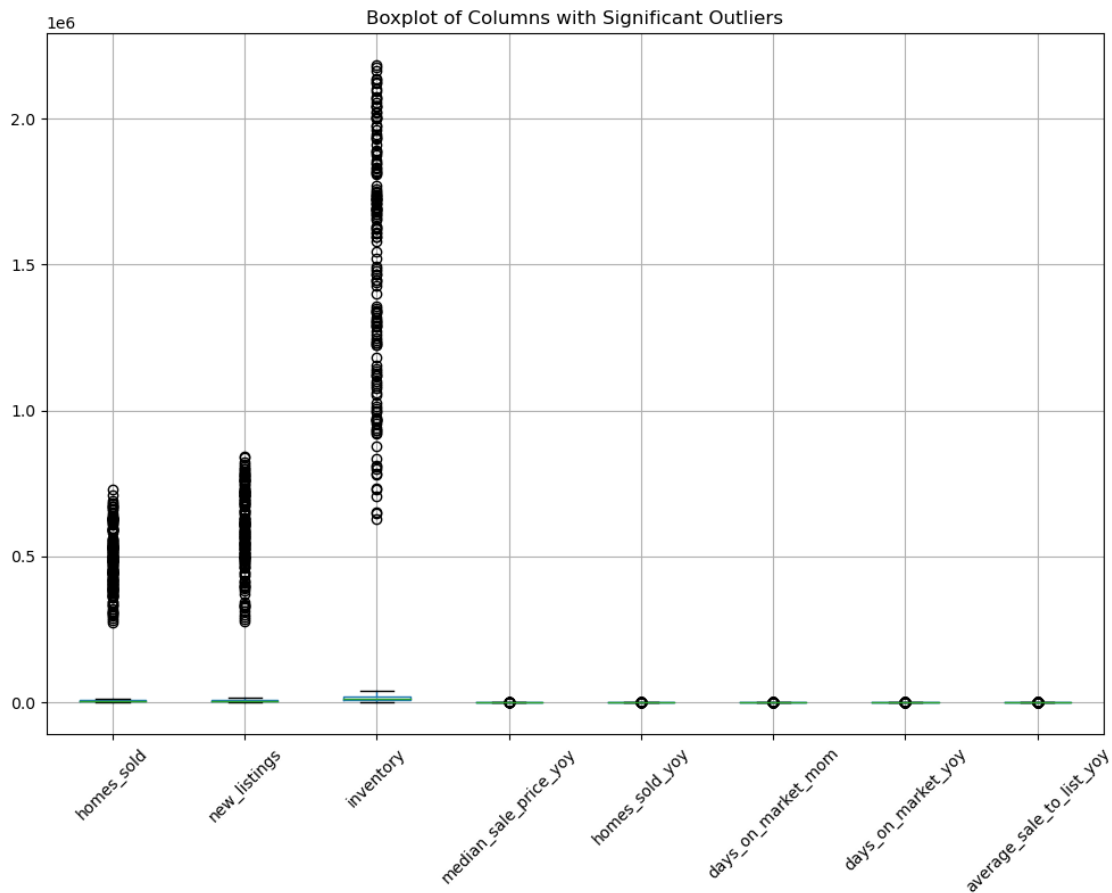
Outlier Insights

Several columns have notable outliers, particularly in: - `homes_sold` (157 outliers), `new_listings` (157 outliers), and `inventory` (157 outliers): These metrics likely have extreme values in certain months or regions. - `median_sale_price_yoy` (37 outliers) and `homes_sold_yoy` (62 outliers): High year-over-year changes indicate significant fluctuations in the market. - `days_on_market_mom` (80 outliers) and `days_on_market_yoy` (75 outliers): The time a home stays on the market shows variability. - `average_sale_to_list_yoy` (97 outliers): Changes in sale-to-list price ratios suggest market volatility.

```
[ ]: # Selected columns with significant outliers
outlier_cols = ["homes_sold",
                "new_listings",
                "inventory",
                "median_sale_price_yoy",
```

```
"homes_sold_yoy",
"days_on_market_mom",
"days_on_market_yoy",
"average_sale_to_list_yoy"]
```

```
[16]: # Visualizing Outliers
e.visualize_outliers(redfin_df, outlier_cols)
```



Boxplot of Columns with Significant Outliers Insights

The boxplot confirms the presence of extreme outliers, particularly in `homes_sold`, `new_listings`, and `inventory`. These could be due to market fluctuations or anomalies in reporting.

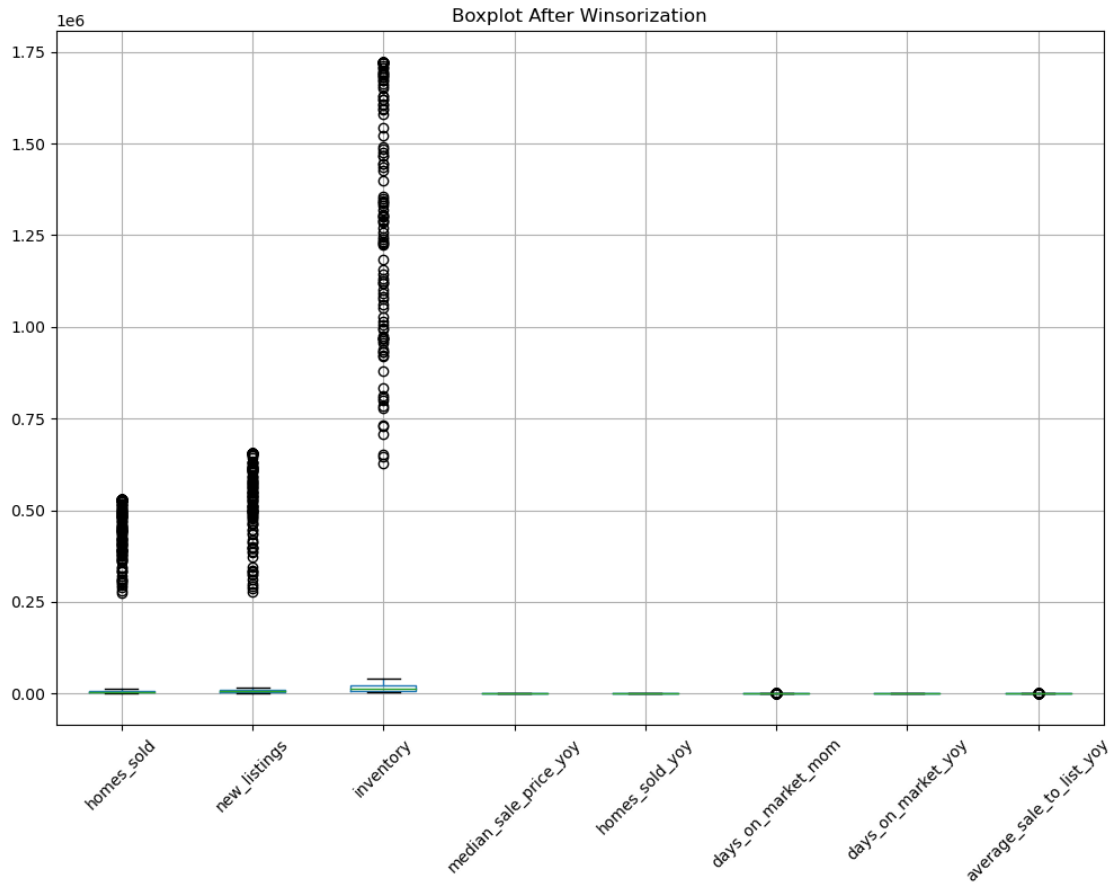
Handling Strategies:

1. Winsorization: Capping extreme values at the 5th and 95th percentiles to reduce impact.
2. Log Transformation: Applying a log transformation to reduce skewness (useful if data is highly right-skewed).

3. Outlier Removal: If values seem unrealistic or errors, we can remove them.

I'll apply Winsorization (capping extreme values) for better distribution while preserving trends.

```
[36]: df_winsorized = e.apply_winsorization(redfin_df, outlier_cols)
```



Boxplot After Winsorization Insight

After Winsorization, the extreme outliers have been capped, making the distribution more balanced while retaining market trends.

Why Are We Creating a Correlation Matrix Again?

The goal of this step is to **reassess the relationships between features after Winsorization**, ensuring that our data transformations have not significantly altered key correlations.

- **Why It Matters:**

- Winsorization **caps extreme values** to reduce the impact of outliers. This can subtly shift correlations between variables.
- We need to confirm whether the **same strong relationships still exist** after handling outliers.
- If correlations remain unchanged, it validates that **Winsorization preserved overall trends** while mitigating the effects of extreme values.
- If correlations change significantly, we may need to revisit **feature selection** and **data transformations** to ensure model reliability.

By computing the correlation matrix post-Winsorization, we verify that our dataset remains **structurally sound for modeling and analysis**.

```
[22]: # Compute correlation matrix
      correlation_matrix = df_winsorized.corr()
```

```
[23]: from IPython.display import display
      display(correlation_matrix)
```

	median_sale_price	median_sale_price_mom	\
median_sale_price	1.000000	0.005228	
median_sale_price_mom	0.005228	1.000000	
median_sale_price_yoy	0.098092	0.178894	
homes_sold	-0.232658	0.014147	
homes_sold_mom	0.001816	0.653534	
homes_sold_yoy	-0.265236	0.077921	
new_listings	-0.237221	0.018802	
new_listings_mom	-0.012778	0.015628	
new_listings_yoy	-0.076005	0.053991	
inventory	-0.270464	-0.002811	
inventory_mom	0.079763	0.205451	
inventory_yoy	0.171810	-0.101212	
days_on_market	-0.472584	-0.075931	
days_on_market_mom	0.036927	-0.626741	
days_on_market_yoy	0.236736	-0.103519	
average_sale_to_list	0.758161	0.111648	
average_sale_to_list_mom	-0.055247	0.556545	
average_sale_to_list_yoy	-0.067936	0.129414	

	median_sale_price_yoy	homes_sold	homes_sold_mom	\
median_sale_price	0.098092	-0.232658	0.001816	
median_sale_price_mom	0.178894	0.014147	0.653534	
median_sale_price_yoy	1.000000	0.057881	0.022894	
homes_sold	0.057881	1.000000	0.014527	
homes_sold_mom	0.022894	0.014527	1.000000	
homes_sold_yoy	0.324370	0.024509	0.127307	
new_listings	0.052116	0.985035	0.018705	

new_listings_mom	-0.018807	-0.041356	-0.024779
new_listings_yoy	0.295929	0.018205	0.114331
inventory	0.020582	0.954897	-0.004080
inventory_mom	0.057880	0.000578	0.234952
inventory_yoy	-0.285196	0.000890	-0.047563
days_on_market	-0.276977	0.120669	-0.090630
days_on_market_mom	-0.069687	-0.032259	-0.608613
days_on_market_yoy	-0.418244	-0.048649	-0.031243
average_sale_to_list	0.387972	-0.084859	0.113571
average_sale_to_list_mom	0.090790	-0.003010	0.496990
average_sale_to_list_yoy	0.610831	-0.019541	0.052232

	homes_sold_yoy	new_listings	new_listings_mom \
median_sale_price	-0.265236	-0.237221	-0.012778
median_sale_price_mom	0.077921	0.018802	0.015628
median_sale_price_yoy	0.324370	0.052116	-0.018807
homes_sold	0.024509	0.985035	-0.041356
homes_sold_mom	0.127307	0.018705	-0.024779
homes_sold_yoy	1.000000	0.024207	0.008135
new_listings	0.024207	1.000000	-0.004320
new_listings_mom	0.008135	-0.004320	1.000000
new_listings_yoy	0.587626	0.022839	0.091386
inventory	0.039470	0.958168	-0.025606
inventory_mom	-0.069369	0.032035	0.482830
inventory_yoy	-0.336885	0.002229	-0.013579
days_on_market	0.130505	0.134936	0.180043
days_on_market_mom	-0.079609	-0.052766	-0.061885
days_on_market_yoy	-0.532169	-0.050926	0.004886
average_sale_to_list	-0.161861	-0.090729	-0.035927
average_sale_to_list_mom	0.170576	0.010565	0.215508
average_sale_to_list_yoy	0.567886	-0.019728	0.003995

	new_listings_yoy	inventory	inventory_mom \
median_sale_price	-0.076005	-0.270464	0.079763
median_sale_price_mom	0.053991	-0.002811	0.205451
median_sale_price_yoy	0.295929	0.020582	0.057880
homes_sold	0.018205	0.954897	0.000578
homes_sold_mom	0.114331	-0.004080	0.234952
homes_sold_yoy	0.587626	0.039470	-0.069369
new_listings	0.022839	0.958168	0.032035
new_listings_mom	0.091386	-0.025606	0.482830
new_listings_yoy	1.000000	0.021441	0.144859
inventory	0.021441	1.000000	-0.002156
inventory_mom	0.144859	-0.002156	1.000000
inventory_yoy	-0.085640	0.013893	0.087980
days_on_market	-0.058362	0.207376	-0.148243
days_on_market_mom	-0.044884	-0.019536	-0.328943
days_on_market_yoy	-0.303271	-0.054352	0.006764

average_sale_to_list	0.026335	-0.155334	0.243446
average_sale_to_list_mom	0.131237	-0.012001	0.200081
average_sale_to_list_yoy	0.397182	-0.038531	0.029244

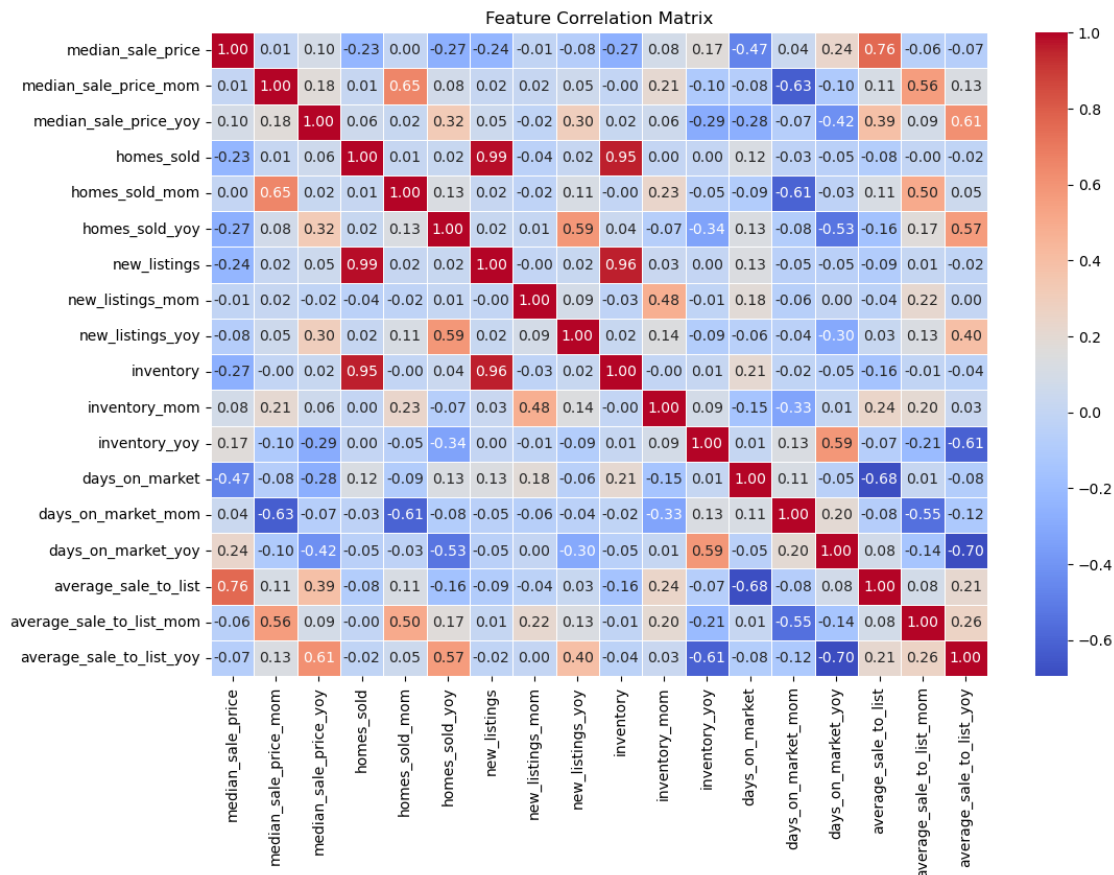
	inventory_yoy	days_on_market	days_on_market_mom \
median_sale_price	0.171810	-0.472584	0.036927
median_sale_price_mom	-0.101212	-0.075931	-0.626741
median_sale_price_yoy	-0.285196	-0.276977	-0.069687
homes_sold	0.000890	0.120669	-0.032259
homes_sold_mom	-0.047563	-0.090630	-0.608613
homes_sold_yoy	-0.336885	0.130505	-0.079609
new_listings	0.002229	0.134936	-0.052766
new_listings_mom	-0.013579	0.180043	-0.061885
new_listings_yoy	-0.085640	-0.058362	-0.044884
inventory	0.013893	0.207376	-0.019536
inventory_mom	0.087980	-0.148243	-0.328943
inventory_yoy	1.000000	0.012950	0.131862
days_on_market	0.012950	1.000000	0.114098
days_on_market_mom	0.131862	0.114098	1.000000
days_on_market_yoy	0.585719	-0.052597	0.196396
average_sale_to_list	-0.067417	-0.684080	-0.078461
average_sale_to_list_mom	-0.211184	0.014321	-0.551240
average_sale_to_list_yoy	-0.611348	-0.082182	-0.118623

	days_on_market_yoy	average_sale_to_list \
median_sale_price	0.236736	0.758161
median_sale_price_mom	-0.103519	0.111648
median_sale_price_yoy	-0.418244	0.387972
homes_sold	-0.048649	-0.084859
homes_sold_mom	-0.031243	0.113571
homes_sold_yoy	-0.532169	-0.161861
new_listings	-0.050926	-0.090729
new_listings_mom	0.004886	-0.035927
new_listings_yoy	-0.303271	0.026335
inventory	-0.054352	-0.155334
inventory_mom	0.006764	0.243446
inventory_yoy	0.585719	-0.067417
days_on_market	-0.052597	-0.684080
days_on_market_mom	0.196396	-0.078461
days_on_market_yoy	1.000000	0.078648
average_sale_to_list	0.078648	1.000000
average_sale_to_list_mom	-0.141950	0.080975
average_sale_to_list_yoy	-0.695177	0.214099

	average_sale_to_list_mom	average_sale_to_list_yoy
median_sale_price	-0.055247	-0.067936
median_sale_price_mom	0.556545	0.129414
median_sale_price_yoy	0.090790	0.610831

homes_sold	-0.003010	-0.019541
homes_sold_mom	0.496990	0.052232
homes_sold_yoy	0.170576	0.567886
new_listings	0.010565	-0.019728
new_listings_mom	0.215508	0.003995
new_listings_yoy	0.131237	0.397182
inventory	-0.012001	-0.038531
inventory_mom	0.200081	0.029244
inventory_yoy	-0.211184	-0.611348
days_on_market	0.014321	-0.082182
days_on_market_mom	-0.551240	-0.118623
days_on_market_yoy	-0.141950	-0.695177
average_sale_to_list	0.080975	0.214099
average_sale_to_list_mom	1.000000	0.261101
average_sale_to_list_yoy	0.261101	1.000000

```
[30]: e.plot_correlation_heatmap(redfin_df)
```



Key Findings from Correlation Analysis

Since Winsorization did not significantly impact the correlation structure, it suggests that the

extreme values, while present, were not highly influential in distorting the relationships between variables. This means the dataset's original trends were robust even with the outliers.

- The high correlation between `homes_sold` and `new_listings` (0.985) suggests that one of these could be dropped without significant information loss.
- `inventory` remains strongly correlated with `homes_sold` (0.955), implying potential redundancy.
- Market trends still hold:
 - Higher prices → Faster sales (Negative correlation between `median_sale_price` & `days_on_market`: -0.473).
 - Price increases reduce days on market (Negative correlation between `median_sale_price_yoy` & `days_on_market_yoy`: -0.418).
 - Sale-to-list ratio trends with median prices (0.611 correlation between `average_sale_to_list_yoy` & `median_sale_price_yoy`).

Feature Engineering Feature Engineering Discussion

The goal of this step is to create meaningful features that enhance the dataset's ability to capture market trends and improve model performance. By engineering new features, we aim to make patterns more interpretable and informative for downstream analysis.

- Market Demand Ratio (`homes_sold` / `new_listings`)
 - Helps measure market competitiveness by indicating how quickly homes are being sold relative to new listings.
 - A higher ratio suggests a strong seller's market, while a lower ratio may indicate higher inventory relative to demand.
- Days on Market Categorization (Fast, Moderate, Slow)
 - Transforms `days_on_market` from a raw numeric value into categories that are easier to analyze.
 - This allows us to better understand how quickly homes are selling and whether certain market conditions lead to longer selling times.

By performing feature engineering at this stage, we are ensuring that the dataset is not just clean, but also structured in a way that makes it more valuable for analysis and predictive modeling.

```
[33]: # Feature Engineering
df_transformed = e.create_features(df_winsorized)
df_transformed.head()
```

```
[33]:
```

	region	month_of_period_end	median_sale_price	median_sale_price_mom	\
0	National	2022-07-01	413000	-3.6	
1	National	2022-08-01	407000	-1.5	
2	National	2022-06-01	429000	-0.7	
3	National	2021-08-01	381000	-1.1	
4	National	2021-09-01	377000	-1.0	

	median_sale_price_yoy	homes_sold	homes_sold_mom	homes_sold_yoy	\
0	7.4	530476	-14.4	-21.9	
1	6.9	530476	5.4	-16.9	
2	10.8	530476	4.1	-15.0	
3	16.2	530476	-1.0	0.1	
4	13.9	530476	-5.1	-4.7	

	new_listings	new_listings_mom	...	inventory_yoy	days_on_market	\
0	656191	-14.3	...	27.1	21	
1	622537	-11.7	...	26.7	27	
2	656191	6.4	...	28.1	18	
3	656191	-9.8	...	-18.2	17	
4	656191	-6.5	...	-16.1	19	

	days_on_market_mom	days_on_market_yoy	average_sale_to_list	\
0	3	6	101.0	
1	5	10	99.9	
2	1	3	102.2	
3	2	-15	101.6	
4	2	-11	101.0	

	average_sale_to_list_mom	average_sale_to_list_yoy	price_bin	\
0	-1.2	-1.2	High	
1	-1.1	-1.7	High	
2	-0.8	-0.3	High	
3	-0.6	2.1	High	
4	-0.6	1.7	High	

	market_demand_ratio	days_on_market_category
0	0.808417	Moderate
1	0.852120	Moderate
2	0.808417	Moderate
3	0.808417	Moderate
4	0.808417	Moderate

[5 rows x 23 columns]

Feature Selection Feature Selection and Reduction Discussion

The goal here is to reduce redundancy in the dataset by removing highly correlated features while retaining the most informative ones for modeling.

- Identifying Redundant Features From our correlation matrix:
 - homes_sold vs. new_listings (0.985 correlation)
 - * Implication: Almost a direct relationship—regions with higher home sales tend to have higher new listings.
 - * Decision: Drop one—likely new_listings because homes_sold directly reflects actual transactions rather than just availability.

- inventory vs. homes_sold (0.955 correlation)
 - * Implication: Inventory reflects available homes, but homes sold captures actual transactions.
 - * Decision: Drop one—likely inventory since homes_sold is more action-based.

```
[35]: # Drop redundant features: 'new_listings' and 'inventory'
df_reduced = e.drop_redundant_features(df_winsorized, ['new_listings',
↪ 'inventory'])

# Display the updated dataset after feature reduction
df_reduced.head()
```

```
[35]:      region month_of_period_end  median_sale_price  median_sale_price_mom \
0   National      2022-07-01          413000          -3.6
1   National      2022-08-01          407000          -1.5
2   National      2022-06-01          429000          -0.7
3   National      2021-08-01          381000          -1.1
4   National      2021-09-01          377000          -1.0

      median_sale_price_yoy  homes_sold  homes_sold_mom  homes_sold_yoy \
0              7.4      530476          -14.4          -21.9
1              6.9      530476           5.4          -16.9
2             10.8      530476           4.1          -15.0
3             16.2      530476          -1.0           0.1
4             13.9      530476          -5.1          -4.7

      new_listings_mom  new_listings_yoy  ...  inventory_yoy  days_on_market \
0             -14.3          -11.9  ...           27.1           21
1             -11.7          -13.7  ...           26.7           27
2              6.4           1.0  ...           28.1           18
3             -9.8           0.0  ...          -18.2           17
4             -6.5          -4.5  ...          -16.1           19

      days_on_market_mom  days_on_market_yoy  average_sale_to_list \
0              3              6              101.0
1              5             10              99.9
2              1              3              102.2
3              2             -15              101.6
4              2             -11              101.0

      average_sale_to_list_mom  average_sale_to_list_yoy  price_bin \
0              -1.2              -1.2          High
1              -1.1              -1.7          High
2              -0.8              -0.3          High
3              -0.6               2.1          High
4              -0.6               1.7          High
```

	market_demand_ratio	days_on_market_category
0	0.808417	Moderate
1	0.852120	Moderate
2	0.808417	Moderate
3	0.808417	Moderate
4	0.808417	Moderate

[5 rows x 21 columns]

3.3.2 Week 4:

- Tasks:
 - Select baseline models for initial testing.
 - Train and evaluate simple models.
- Goal: Identify the best initial modeling approach.

[]:

[]:

[]:

[]:

[]:

[]:

[]:

[]:

[]:

[]: